

King Salman International University



Intelligent Robot for Flame Location Detection

Phoenix Hardware & Electronics

Bassel Ashraf Ahmed Elbahnasy

221101208

Supervisor: Dr. Mahmoud Zaki

Program Director: Dr. Ayman Elshabrawy

Faculty: Computer Science & Engineering

Program: Computer Engineering

Abstract

This report details the hardware architecture, electrical circuit design, and software integration of an Intelligent Mobile Robot designed for autonomous flame location detection and suppression. Built on a 4-wheel drive skid-steer chassis, the physical platform is driven by high-torque JGB37-545 DC geared motors controlled via dual BTS7960 motor drivers, and is powered by a robust 22.2V Li-Po dual-battery system. The core onboard computational engine is a Raspberry Pi 4 Model B, which handles local navigation, sensor data acquisition, and actuator control.

To optimize power efficiency and maintain real-time responsiveness, the system employs a distributed split-node architecture. A stationary external unit acts as the global vision node, utilizing a TP-Link Tapo C210 camera and dual custom Convolutional Neural Networks (CNNs) to identify fire sources and prioritize the detection of humans in danger.

The mobile edge node (Raspberry Pi 4 Model B) uses an Okdo LiDAR HAT for 360-degree obstacle detection and a Raspberry Pi Camera Module V3 for local perception. By computing real-time extrinsic transformation matrices using OpenCV and ArUco fiducial markers, the robot maps the 2D visual data into a 3D physical workspace. This allows the ROS 2 Navigation Stack (Nav2) to dynamically generate collision-free paths to the fire source. Upon successful arrival, the system autonomously triggers a 24V water pump via an opto-isolated relay to extinguish the flame. This document provides a comprehensive specification of the selected components, kinematic models, power distribution circuitry, and the ongoing development roadmap toward full autonomous operation. The system's autonomous navigation and collision avoidance logic were successfully validated through high-fidelity Gazebo Harmonic simulations, utilizing custom occupancy grids and tuned friction parameters to ensure reliable skid-steer performance.

Table of Contents

Abstract	2
1. Components	7
1.1. Raspberry Pi 4 Model B	7
1.1.1 Overview and Role in the System.....	7
1.1.2 Processing and Computational Capabilities	7
1.1.3 Memory Subsystem	7
1.1.4 Graphics and Multimedia Support	7
1.1.5 Connectivity and Communication Interfaces	8
1.1.6 I/O Ports and Expansion Capabilities	8
1.1.7 Power Management	8
1.1.8 Software Ecosystem	8
1.1.9 Justification for Selection	9
1.2. Raspberry Pi Camera Module V3	9
1.2.1 Overview and Role in the System.....	9
1.2.2 Image Sensor Characteristics	9
1.2.3 Optics and Autofocus System	10
1.2.4 Imaging and Video Capabilities	10
1.2.5 Field of View and Variants	11
1.2.6 Interface and Communication	11
1.2.7 Physical Characteristics and Integration	11
1.2.8 Justification for Selection	11
1.3. Okdo LiDAR	12
1.3.1 Overview and Role in the System.....	12
1.3.2 LiDAR Technology and Operating Principle	12
1.3.3 Distance Measurement and Accuracy	13
1.3.4 Scanning Performance	13
1.3.5 Interface and Communication	13
1.3.6 Physical Characteristics and Integration	14

1.3.7 Justification for Selection	14
1.4. TP-Link Tapo C210	15
1.4.1 Overview and Role in the System.....	15
1.4.2 Imaging and Video Capabilities	15
1.4.3 Pan and Tilt Mechanism	16
1.4.4 Night Vision and Low-Light Performance	16
1.4.5 Audio System	16
1.4.6 Network and Communication Interfaces	16
1.4.7 Power and Hardware Characteristics.....	17
1.4.8 Physical Characteristics and Installation	17
1.4.9 Software Support and Compatibility.....	17
1.5. Lithium Polymer Battery (5200 mAH)	18
1.5.1 Selection Strategy and Role	18
1.5.2 Justification of Electrical Characteristics	18
1.6. XL4015 Adjustable DC-DC Step-Down.....	19
1.6.1 Selection Strategy and Role	19
1.6.2 Justification of Capabilities.....	19
1.7. IMAX B6 Balancer Charger	20
1.7.1 Selection Strategy and Role	20
1.7.2 Justification of Capabilities.....	20
1.8. Motor Driver BTS7960	21
1.8.1 Selection Strategy and Role	21
1.8.2 Justification of Capabilities.....	22
2. Electric Circuit Design	22
2.1. Power Source Configuration	22
2.2. Main Power Bus	22
2.3. Motor Drivers and Actuators	23
2.4. Voltage Regulation and Logic Power	23
2.5. Grounding Scheme	23

2.6.	Design Considerations	23
2.7.	Summary of Voltage Paths	24
2.8.	Power Distribution System	24
2.8.1.	Voltage Rails	24
3.	System Architecture and Operational Overview	25
3.1.	Executive Summary of Electrical Architecture	25
4.	System Architecture	26
4.1.	Split-Node Operational Model	26
4.1.1.	Vision Node (Stationary Laptop)	26
4.1.2.	Mobile Edge Node (Raspberry Pi 4 Model B)	26
4.1.3.	Communication Layer	26
5.	Computing and Control Hardware	28
5.1.	Raspberry Pi 4 Model B — Mobile Control Unit	28
5.2.	Laptop — Vision Processing Unit	28
6.	Actuators and Drive Electronics	29
6.1.	Locomotion System	29
6.2.	Steering System	29
6.3.	Fire Suppression System	29
7.	Sensors and Perception Layer	29
7.1.	Global Detection — TP-Link Tapo C210	29
7.2.	Local Navigation — Okdo LiDAR HAT	30
8.	Communication and Logic Flow	30
8.1.	MQTT Workflow	30
9.	Interface and Wiring Logic	30
9.1.	Raspberry Pi 4 Model B GPIO Allocation	30
9.2.	Logic-Level Compatibility and Protection	33
9.2.1.	Motor Driver Enable Logic	33
9.2.2.	PWM Control Signals	34
9.2.3.	Relay Control for Pump and Solenoid	34

9.2.4. LiDAR HAT Interface	34
9.3. Grounding Strategy.....	34
9.4. High-Current Wiring Considerations	35
10. Kinematic and Transformation Modeling.....	35
10.1. System Parameters	35
10.2. Inverse Kinematics (Velocity Commands to Wheel Speeds)	35
10.3. Forward Kinematics (Wheel Speeds to Robot Motion).....	36
10.4. Homogeneous Transformation Matrices (TF)	37
10.5. Extrinsic Transformation Modeling (Surveillance Camera to Robot Frame)	37
10.5.1. The Python Implementation (OpenCV)	38
11. ROS 2 System Integration & Control	47
11.1. Phase 1: Simulation and Robot Modeling (Gazebo Harmonic & WSL Environment)	47
11.2. Phase 2: Hardware Control (Raspberry Pi 4 Model B)	60
11.3. Phase 3: Actuator, Sensor, and Communication Nodes	63
11.3.1. Pump Controller Node (pump_controller.py)	63
11.3.2. LiDAR Publisher Node (lidar_publisher.py)	65
11.3.3. MQTT to Nav2 Action Client (mqtt_nav_client.py)	67
11.4: System Dependencies	70
12. Current Progress and Future Work	70
12.1. Current Development Status	70
12.2. Future Work Roadmap	71
13. Conclusion	72
References	72
Appendix A: Bill & Total Cost	73

1. Components

1.1. Raspberry Pi 4 Model B



1.1.1 Overview and Role in the System

The Raspberry Pi 4 Model B serves as the primary mobile control and edge-processing unit of the intelligent fire-detection robot. Following a hardware revision to improve electrical isolation, this board handles real-time motor control, sensor data acquisition, local decision-making, and communication with the external vision processing node. The board was selected due to its proven reliability, computational performance, extensive I/O support, and strong software ecosystem, making it highly suitable for embedded robotics applications.

1.1.2 Processing and Computational Capabilities

The board is powered by a Broadcom BCM2711, Quad-core Cortex-A72 (ARM v8) 64-bit SoC operating at 1.5 GHz. This architecture easily handles the ROS 2 Navigation Stack and local MQTT broker without bottlenecks, enabling the execution of concurrent tasks such as motor control loops, sensor polling, and communication handling.

1.1.3 Memory Subsystem

The Raspberry Pi 4 utilizes LPDDR4-3200 SDRAM. This memory capacity allows stable multitasking, buffering of sensor data streams, and execution of communication services such as MQTT brokers or clients without memory exhaustion.

1.1.4 Graphics and Multimedia Support

The Raspberry Pi 4 incorporates the VideoCore VI GPU. While advanced graphics rendering is not a primary requirement for this project, the GPU enables efficient camera handling

and video decoding. The board supports dual HDMI outputs (up to 4K resolution) and hardware-accelerated HEVC video decoding.

1.1.5 Connectivity and Communication Interfaces

The Raspberry Pi 4 provides multiple communication interfaces essential for robotic systems:

- **Wireless Connectivity:** Dual-band IEEE 802.11ac Wi-Fi and Bluetooth 5.0 with Low Energy (BLE).
- **Wired Networking:** Gigabit Ethernet.

These interfaces enable reliable communication between the mobile robot and the stationary vision processing node, as well as remote monitoring and control.

1.1.6 I/O Ports and Expansion Capabilities

The board offers extensive expansion options through:

- **USB Interfaces:** 2 x USB 3.0 ports and 2 x USB 2.0 ports.
- **Camera and Display Interfaces:** 2-lane MIPI CSI/DSI transceivers.
- **GPIO:** Standard 40-pin GPIO header, used for motor driver control, relay activation, and sensor interfacing.

This flexibility allows seamless integration of actuators, sensors, and peripheral modules required by the robotic platform.

1.1.7 Power Management

The Raspberry Pi 4 Model B is powered via its USB-C interface, requiring a 5.1V at 3A input. To prevent brownouts and fatal voltage spikes caused by inductive loads, the Pi is powered by an independent, external USB-C power bank rather than the robot's 22.2V main bus. This ensures the microcontroller receives clean, isolated power, safely decoupled from the electrical noise of the servos and motors. However, to maintain accurate PWM signaling and logic control, a single ground wire connects the Raspberry Pi to the main chassis ground, ensuring a shared 0V reference across all isolated power supplies.

1.1.8 Software Ecosystem

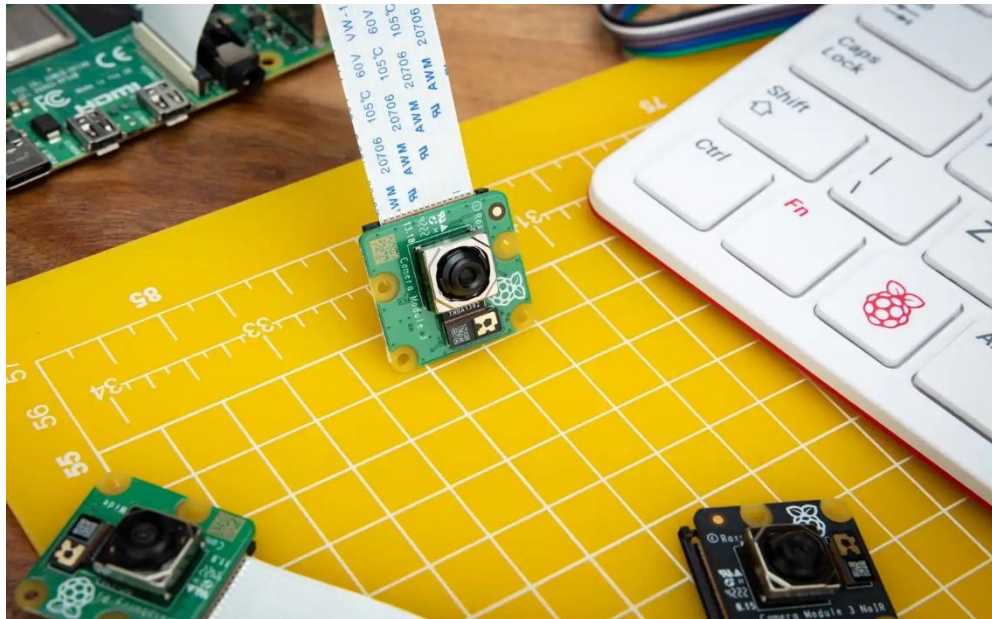
The Raspberry Pi 4 supports multiple Linux-based operating systems, including Ubuntu, providing access to a mature development environment. The availability of extensive libraries for GPIO control, networking, camera handling, and robotics frameworks significantly reduces development complexity and accelerates system integration.

1.1.9 Justification for Selection

The Raspberry Pi 4 Model B was selected over microcontroller-based solutions due to:

- Superior processing capability for multitasking and ROS 2 navigation.
- Native support for high-bandwidth sensors and networking.
- Strong community and long-term software support.
- Proven reliability in mobile robotics environments under heavy load.

1.2. Raspberry Pi Camera Module V3



1.2.1 Overview and Role in the System

The Raspberry Pi Camera Module V3 is employed as the **primary onboard visual sensing device** for the mobile robotic platform. It provides real-time image and video acquisition to support local perception tasks, system monitoring, and potential future onboard vision processing. The module was selected due to its **high image quality, native compatibility with the Raspberry Pi ecosystem, compact form factor, and low power consumption**, making it well-suited for embedded robotic applications.

1.2.2 Image Sensor Characteristics

The Camera Module V3 is based on the **Sony IMX708** image sensor, a back-illuminated stacked CMOS sensor designed for high sensitivity and low noise performance. The sensor

offers a **native resolution of 11.9 megapixels**, with an effective pixel array of **4608 × 2592 pixels** and a pixel size of **1.4 μm × 1.4 μm**.

Key sensor features include:

- High signal-to-noise ratio (SNR)
- Built-in **2D dynamic defect pixel correction**
- Improved low-light performance compared to previous camera generations.

These characteristics enable reliable image capture under varying lighting conditions, which is essential for fire detection and environmental monitoring scenarios.

1.2.3 Optics and Autofocus System

The Camera Module V3 integrates a **Phase Detection Autofocus (PDAF)** system, allowing fast and accurate focus adjustment across different object distances. Autofocus control is managed through a **two-wire serial interface**, enabling software-based focus tuning.

The optical system supports:

- Rapid focus acquisition
- Stable focus tracking during motion
- Improved image sharpness for both near-field and mid-range targets

This capability is particularly beneficial for mobile robotics, where camera-to-object distance continuously changes during navigation.

1.2.4 Imaging and Video Capabilities

The camera module supports both still image capture and high-frame-rate video streaming. Common supported video modes include:

- **1080p at up to 50 frames per second**
- **720p at up to 100 frames per second**
- **480p at up to 120 frames per second**

Image data is output in **RAW10 format**, allowing flexible post-processing and efficient integration with computer vision pipelines. The module also supports **High Dynamic Range (HDR)** imaging, improving visibility in scenes with strong brightness contrast, such as fire sources against darker backgrounds.

1.2.5 Field of View and Variants

The standard variant of the Camera Module V3 provides a **diagonal field of view of approximately 75°**, offering a balanced trade-off between scene coverage and spatial resolution. This field of view is suitable for forward-facing perception tasks, obstacle awareness, and visual confirmation of detected fire sources.

A NoIR variant is also available, which omits the infrared cut filter for infrared-based imaging; however, the standard variant was selected for this project due to its suitability for visible-light fire detection.

1.2.6 Interface and Communication

The Camera Module V3 connects to the Raspberry Pi 4 Model B via a **CSI-2 (Camera Serial Interface)**, providing a high-bandwidth, low-latency data path for image transmission. Control communication is handled through a **two-wire serial interface**, supporting fast and reliable configuration of camera parameters.

The module uses a **15-pin flexible flat cable (FPC)** with a 1 mm pitch, ensuring secure mechanical connection while maintaining a compact footprint.

1.2.7 Physical Characteristics and Integration

The camera module features a compact and lightweight design, with dimensions suitable for direct mounting on mobile robotic platforms. Its small size minimizes mechanical interference and allows flexible placement to optimize the robot's visual field.

The module is powered directly from the Raspberry Pi through the CSI interface, eliminating the need for additional power regulation circuitry and simplifying system integration.

1.2.8 Justification for Selection

The Raspberry Pi Camera Module V3 was selected over third-party USB cameras due to:

- Native hardware and software compatibility with the Raspberry Pi 4 Model B
- Lower latency and reduced CPU overhead via CSI-2 interface
- Superior image quality and autofocus capability.
- Compact form factor and simplified power requirements

These advantages make the Camera Module V3 an effective and reliable visual sensing solution for an intelligent robotic fire detection system.

1.3. Okdo LiDAR



1.3.1 Overview and Role in the System

The Okdo LiDAR HAT is used as the **primary local navigation and obstacle-detection sensor** for the mobile robotic platform. It provides real-time distance measurements across a full 360-degree field, enabling the robot to perceive its immediate surroundings, avoid obstacles, and support autonomous movement toward detected fire sources. The module was selected due to its **compact design, direct compatibility with single-board computers, and reliable ranging performance** in indoor environments.

1.3.2 LiDAR Technology and Operating Principle

The Okdo LiDAR HAT integrates the **LD06 LiDAR sensor**, which operates using **Direct Time-of-Flight (DToF)** technology. In this method, an infrared laser pulse is emitted toward surrounding objects, and the distance is calculated by measuring the time taken for the reflected signal to return to the sensor.

This approach provides:

- Accurate distance measurements independent of ambient lighting
- Stable performance in low-visibility environments
- High reliability for indoor robotic navigation

DTOF-based LiDAR is particularly suitable for fire-response robotics, where lighting conditions may vary significantly.

1.3.3 Distance Measurement and Accuracy

The LD06 sensor offers a measurement range suitable for indoor navigation tasks:

- **Minimum detectable distance:** Approximately 2 cm (Hardware limit); however, for system integration, this is software-calibrated to 0.35 m to prevent the sensor from detecting the robot's own chassis and camera housing as obstacles
- **Maximum detectable distance:** up to 10 m

The sensor provides a **ranging accuracy of approximately ± 30 mm**, which is sufficient for obstacle avoidance, corridor navigation, and spatial awareness in confined environments. A sampling rate of approximately **4500 points per second** ensures dense spatial data acquisition.

1.3.4 Scanning Performance

The LiDAR module performs continuous **360-degree rotational scanning**, generating a circular point cloud around the robot. The scanning frequency ranges from **5 Hz to 13 Hz**, allowing a balance between spatial resolution and update rate.

This scanning capability enables:

- Real-time obstacle detection
- Environmental mapping
- Support for navigation algorithms such as wall following or waypoint tracking.

The high angular resolution of the generated point cloud makes the sensor suitable for integration with SLAM or reactive navigation strategies.

1.3.5 Interface and Communication

The Okdo LiDAR HAT communicates with the Raspberry Pi 4 Model B via a **UART serial interface**, transmitting real-time distance and angular data. This communication method offers low latency and minimal processing overhead, making it well-suited for continuous sensor streaming in embedded systems.

The HAT form factor allows direct mounting onto the Raspberry Pi, simplifying wiring and ensuring reliable electrical connections.

1.3.6 Physical Characteristics and Integration

The LiDAR module features a compact mechanical design, with dimensions suitable for installation on a mobile robotic chassis without obstructing other components. Its lightweight construction minimizes impact on the robot's center of gravity and overall mobility.

Power is supplied directly from the host single-board computer, eliminating the need for additional power regulation circuitry and simplifying system integration.

1.3.7 Justification for Selection

The Okdo LiDAR HAT was selected over ultrasonic or infrared proximity sensors due to:

- Full 360-degree environmental coverage
- Higher spatial resolution and accuracy
- Independence from ambient lighting conditions
- Direct compatibility with the Raspberry Pi platform

These characteristics make the LiDAR HAT a reliable and scalable solution for local navigation and obstacle detection in an intelligent robotic fire detection system.

1.4. TP-Link Tapo C210



1.4.1 Overview and Role in the System

The TP-Link Tapo C210 is utilized as a **stationary global vision and monitoring node** within the intelligent fire detection system. Unlike the onboard Raspberry Pi camera, which provides local perception, the Tapo C210 operates as an **external fixed observation unit**, offering a wide-area visual overview of the environment. This separation enables computational offloading and enhances system reliability by providing an independent visual reference for fire detection and situational awareness.

1.4.2 Imaging and Video Capabilities

The Tapo C210 is equipped with a **1/2.8-inch CMOS image sensor** capable of capturing video at a maximum resolution of **2K (2304 × 1296 pixels)**. Video streams are encoded using the **H.264 compression standard**, balancing image quality and bandwidth efficiency.

Key imaging characteristics include:

- High-resolution video suitable for long-range visual monitoring

- Stable frame rate of up to **15 frames per second**
- Adequate image clarity for flame detection and motion analysis

These features allow the camera to function effectively as a global perception node in indoor environments.

1.4.3 Pan and Tilt Mechanism

A defining feature of the Tapo C210 is its integrated **pan-and-tilt mechanism**, which provides:

- **360° horizontal rotation**
- **114° vertical tilt**

This mechanical range enables near-complete coverage of the monitored area using a single camera unit. The camera also supports **automatic motion tracking**, allowing it to follow detected movement within its field of view. This capability enhances situational awareness and reduces blind spots during fire detection and monitoring operations.

1.4.4 Night Vision and Low-Light Performance

The camera incorporates **infrared LEDs operating at 850 nm**, enabling night vision with an effective range of up to **9 meters**. An automatic IR-cut filter switches between day and night modes based on ambient lighting conditions.

This functionality ensures continuous monitoring capability under low-light or smoke-affected conditions, which is critical for fire detection scenarios.

1.4.5 Audio System

The Tapo C210 includes a **two-way audio system**, consisting of a built-in microphone and speaker. While audio communication is not a primary requirement for this project, the feature provides potential future extensions such as:

- Audible alerts
- Remote voice warnings
- Environmental sound monitoring

1.4.6 Network and Communication Interfaces

The Tapo C210 connects to the network using **IEEE 802.11 b/g/n Wi-Fi** operating on the **2.4 GHz band**. It supports standard networking protocols including:

- TCP/IP

- HTTP and HTTPS
- RTSP and ONVIF

These protocols enable seamless integration with external processing systems, allowing video streams to be accessed by the stationary vision processing unit for fire detection and analysis.

1.4.7 Power and Hardware Characteristics

The camera is powered using an external adapter rated at **9 V DC, 0.6 A**, supplied from standard AC mains. This fixed power configuration makes the device suitable for continuous stationary operation without impacting the mobile robot's power budget.

1.4.8 Physical Characteristics and Installation

The Tapo C210 features a compact enclosure designed for indoor use. It supports multiple mounting options, including:

- Desktop placement
- Wall mounting
- Ceiling mounting

These options allow flexible installation to optimize coverage of the monitored environment.

1.4.9 Software Support and Compatibility

The camera is managed through the **Tapo mobile application**, supporting both Android and iOS platforms. It is also compatible with common smart-home ecosystems and voice assistants. For this project, the camera's primary role is video streaming rather than user interaction, and control is handled through network-based access protocols.

1.5. Lithium Polymer Battery (5200 mAh)



1.5.1 Selection Strategy and Role

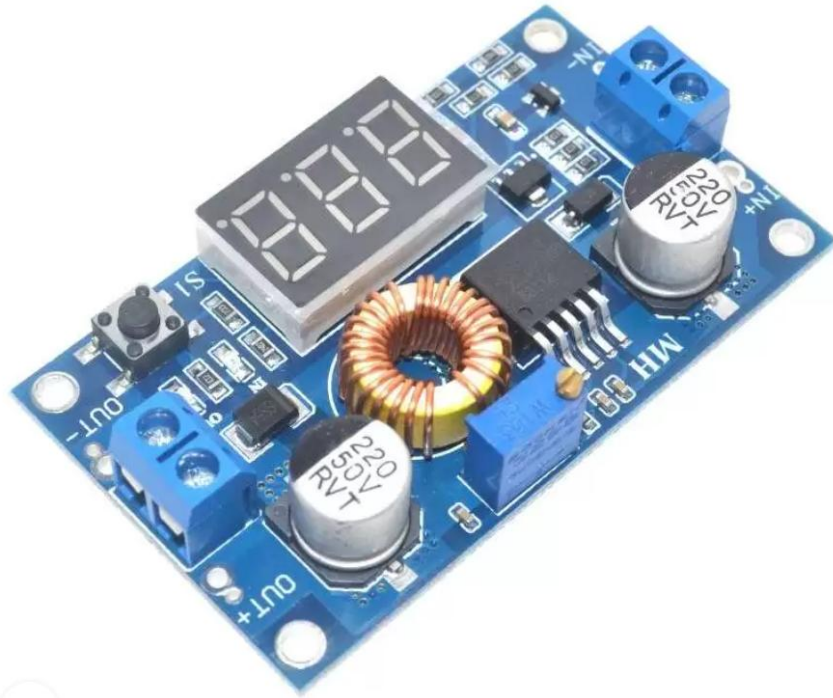
The power source is the backbone of any mobile robotic platform. Two **11.1 V 5200 mAh Lithium Polymer (LiPo) batteries** were selected to form a series configuration, creating a **22.2 V main power bus**. This specific model was chosen over Lead-Acid or Ni-MH alternatives due to its superior energy-to-weight ratio, which is critical for maintaining the robot's mobility and reducing strain on the chassis.

1.5.2 Justification of Electrical Characteristics

- **High Discharge Capability (40C):** The primary reason for selecting this specific battery is its 40C discharge rate. The robot's drivetrain and water pump impose sudden, high-current demands during acceleration or fire suppression. A standard low-discharge battery would suffer from voltage sag under these loads, potentially causing the microcontroller to reset. The 40C rating ensures stable power delivery even during peak load events.
- **Capacity for Extended Runtime:** With a capacity of **5200 mAh**, this battery provides sufficient energy headroom to sustain the Raspberry Pi 4 Model B's processing, Wi-Fi streaming, and motor operation for extended durations without frequent recharging.

- **Voltage Headroom:** Using a 3S1P (11.1 V) configuration allowed for a series connection to reach ~22.2 V. This high input voltage is essential for the buck converters to operate efficiently, as they require an input voltage significantly higher than their target output to maintain stable regulation.

1.6. XL4015 Adjustable DC-DC Step-Down



1.6.1 Selection Strategy and Role

In the revised split-rail power architecture, the XL4015 Step-Down Converter is dedicated exclusively to powering the 5V peripheral components, namely the MG996R pan-tilt servo motors and the Okdo LiDAR HAT. This component is critical because it isolates the electrically "noisy" inductive loads of the servos from the sensitive logic board of the Raspberry Pi 4. By removing the Pi from this rail, the system is protected against brownouts and voltage sags caused by sudden servo movements.

1.6.2 Justification of Capabilities

- **Dedicated Load Handling:** With a 5 A capacity, this module offers significant headroom to handle the sudden current spikes generated when the servos actuate or hold a position.
- **Precision Voltage Regulation:** The XL4015 offers excellent load regulation ($\leq 0.8\%$), ensuring that the voltage remains stable even when the servos draw varying amounts of current.

- **Visual Monitoring:** The integration of an onboard digital voltmeter allows for immediate visual verification of the voltage levels without external multimeters. This feature simplifies debugging and ensures that the voltage is correctly calibrated before connecting sensitive components.
- **Voltage Headroom:** Using a 3S1P (11.1 V) configuration allowed for a series connection to reach ~22.2 V. This high input voltage is essential for the buck converter to operate efficiently, as it requires an input voltage significantly higher than its target 5V output to maintain stable regulation.

1.7. IMAX B6 Balancer Charger



1.7.1 Selection Strategy and Role

The **IMAX B6 Balancer Charger** was included in the project inventory not as an onboard component, but as essential support equipment. Given the use of high-capacity LiPo batteries, a standard "wall-wart" charger would be unsafe. The IMAX B6 was chosen to ensure the longevity and safety of the power system.

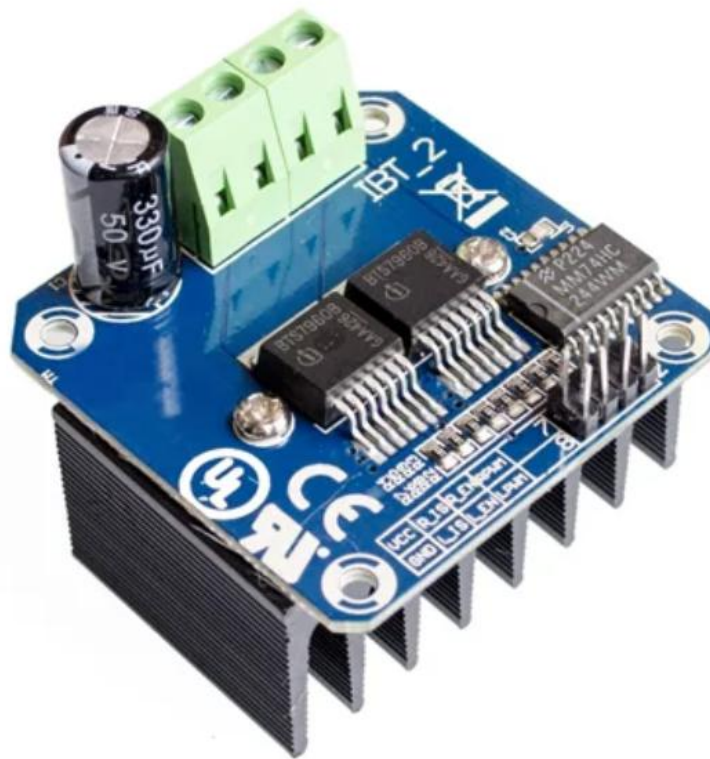
1.7.2 Justification of Capabilities

- **Cell Balancing:** LiPo batteries can become dangerous if the individual cells in the pack drift apart in voltage. The IMAX B6 actively balances the cells during charging,

which is a critical safety requirement for preventing fire hazards and extending battery life.

- **Storage Mode:** The charger includes a dedicated "Storage" function that charges or discharges the battery to an optimal voltage level for long-term inactivity. This is essential for an academic or prototype project where the robot may sit idle for periods between tests, preventing battery chemistry from degrading.
- **Safety Monitoring:** Features such as delta-peak sensitivity and time-limit protection prevent overcharging, ensuring that the charging process remains safe even in a laboratory environment without constant supervision.

1.8. Motor Driver BTS7960



1.8.1 Selection Strategy and Role

For the locomotion system, the **BTS7960 High-Power Motor Driver** was selected to interface the Raspberry Pi with the high-torque DC geared motors. This driver was chosen over the more common L298N module, which is limited to 2 A and would have instantly overheated or failed under the load of this robot's drivetrain. To accommodate the 4WD upgrade, the system utilizes two drivers, with each driver powering two motors in parallel for the left and right sides of the chassis.

1.8.2 Justification of Capabilities

- **High Current Capacity:** The BTS7960 is capable of handling up to **43 A of continuous current**. This massive headroom ensures that the driver operates well within its safe limits, even when the robot is navigating carpeted surfaces or carrying a full payload, which significantly increases motor current draw.
- **Thermal Protection & Reliability:** Unlike basic H-bridges, the BTS7960 features an "Intelligent Power" design with built-in thermal shutdown and over-current protection. This robustness makes it ideal for a fire-fighting robot where reliability in harsh conditions is paramount.
- **Logic Isolation:** The driver includes isolation that allows the 3.3 V logic signals from the Raspberry Pi to safely control the high-voltage motor bus (22.2 V) without the risk of high-voltage spikes damaging the delicate GPIO pins.
- **PWM Efficiency:** The support for high-frequency PWM (up to 25 kHz) allows for smooth speed control and quieter motor operation compared to low-frequency drivers, resulting in more precise navigation and steering control.

2. Electric Circuit Design

This section describes the electrical circuit design implemented in the project, focusing on power distribution, voltage regulation, and component interfacing. The system is engineered to deliver stable and efficient power to multiple subsystems using dual-battery architecture and step-down converters.

2.1. Power Source Configuration

- **Battery Setup:** Two **11.1 V 5200 mAh Li-Po batteries** are connected in **series**, providing a total supply voltage of approximately **22.2 V DC**.
- **Purpose:** This high-voltage configuration ensures sufficient headroom for voltage regulation and supports high-current loads such as motors and pumps.

2.2. Main Power Bus

- The combined 22.2 V output is routed to a **main power bus**, which acts as the central distribution node.
- All downstream components draw power from this bus, either directly or through regulated converters.

2.3. Motor Drivers and Actuators

- **DC Geared Motors (*4):** Four JGB37-545 DC geared motors are powered directly from the 22.2 V bus via Motor Driver 1 and Motor Driver 2. The front-left and back-left motors are wired in parallel to Driver 1, while the front-right and back-right motors are wired in parallel to Driver 2.
- **Water Pump:** The system utilizes a **24 V diaphragm pump**. It is powered directly from the **22.2 V Main Power Bus**, eliminating the need for a step-down converter while ensuring sufficient pressure and flow rate for fire suppression.

2.4. Voltage Regulation and Logic Power

- **Logic Step-Down Converter (XL4015):**
 - Converts 22.2 V to **5 V DC** for low-voltage components.
 - Powers:
 - **Two Servo Motors** (5 V each)
- **Converter Type:**
 - The system utilizes a single **XL4015 DC-DC Buck Converter**, chosen for its high efficiency to regulate the 5V logic rail.

2.5. Grounding Scheme

- A **common ground** is shared across all components to ensure stable voltage references and prevent floating ground issues.
- Ground lines are clearly marked and routed to maintain signal integrity and safety.

2.6. Design Considerations

- **Voltage Matching:**
 - Each component receives voltage tailored to its operating range via dedicated converters.
- **Current Handling:**
 - Motor drivers and converters are rated to handle peak currents without overheating or voltage sag.
- **Safety:**
 - Over-voltage, reverse polarity, and short-circuit protection are integrated into converters and drivers.

- **Modularity:**
 - The design allows easy replacement or upgrade of individual modules without affecting the entire system.

2.7. Summary of Voltage Paths

Source Voltage	Destination	Regulated Voltage	Components Powered
22.2 V (Series Li-Po)	Motor Driver 1 & 2	22.2 V	4x JGB37-545 DC Geared Motors
22.2 V	Relay Module	22.2 V	Water Pump(24V)
22.2 V	Step-Down Converter (XL4015)	5 V	Servo Motors
External Power Bank	Raspberry Pi 4 Model B	5.1 V	

This circuit design ensures robust power delivery across mechanical and computational subsystems, enabling reliable operation of the robotic platform. It reflects careful planning in voltage regulation, current distribution, and modular integration—key principles in embedded system design.

2.8. Power Distribution System

The robot’s power system is engineered around a **22.2 V main bus**, derived from two 11.1 V Li-Po batteries connected in series. This configuration provides:

- High voltage headroom for efficient buck conversion
- Stable supply for high-current motors
- Reduced current draw across wiring, minimizing losses.

2.8.1. Voltage Rails

22.2 V Main Bus

- Directly powers the four high-torque JGB37-545 DC geared motors via dual BTS7960 drivers.
- Directly powers the 24 V high-pressure water pump.

- Provides input to the 5V buck converter.

5 V Rail (Step-Down Converter (XL4015))

- Powers:
 - Pan-tilt servos
- Isolated from motor loads to prevent brownouts.

This layered power system ensures stable operation across all subsystems, even during high-current events such as motor stall or pump activation.

3. System Architecture and Operational Overview

3.1. Executive Summary of Electrical Architecture

The Intelligent Mobile Robot operates on a distributed, two-node architecture designed to balance computational load, power efficiency, and real-time responsiveness. The system separates high-intensity computer vision tasks from onboard navigation and actuation, enabling reliable performance under battery-powered constraints.

The architecture consists of:

- **Node 1—Vision & Processing Station (Laptop):** Performs heavy computer vision tasks using custom CNN models for dual fire and human detection, and streams prioritized target coordinates to the robot.
- **Node 2 — Mobile Edge Node (Raspberry Pi 4 Model B):** Handles navigation, LiDAR processing, actuator control, and real-time decision-making.
- **Communication Layer — Local Mosquitto MQTT Broker:** Provides asynchronous, low-latency messaging between the two nodes.

This distributed design ensures that the robot maintains high responsiveness while minimizing onboard power consumption and thermal load.

4. System Architecture

4.1. Split-Node Operational Model

4.1.1. Vision Node (Stationary Laptop)

- Runs the custom CNN object detection pipelines (Fire Detection & Human Priority overriding).
- Receives RTSP video stream from the TP-Link Tapo C210
- Detect fire and compute its location within the environment.
- Publishes target coordinates to the MQTT broker.

4.1.2. Mobile Edge Node (Raspberry Pi 4 Model B)

- Subscribes to MQTT topics for navigation and fire-location data.
- Processes LiDAR readings for obstacle detection
- Controls motors, servos, pump, and solenoid valve
- Executes the fire-suppression sequence autonomously.

4.1.3. Communication Layer

- **local MQTT Broker (Mosquitto):** Replaces the previous cloud-based HiveMQ setup to handle asynchronous messaging directly over the local Wi-Fi or Ethernet network.
- **Ultra-Low Latency & Reliability:** By eliminating internet dependency and cloud routing, the system ensures near-instantaneous message delivery. This is critical for safety mechanisms (like the heartbeat watchdog) and real-time navigation commands.
- **Topic-Based Publish/Subscribe Structure:** Retains a highly modular communication format, allowing the vision node, navigation client, and hardware controllers to operate independently and making future system expansions simple.
- **Fully Localized Autonomy:** This updated architecture transitions the robot from a hybrid, cloud-dependent IoT setup into a fully localized edge-computing platform, maximizing real-time responsiveness and operational stability in environments without internet access.

Task / Feature	Raspberry Pi 4 Model B (Mobile Edge Node)	External Laptop (Vision Node)
Primary Role	Acts as the mobile onboard controller and edge-processor.	Acts as the stationary global vision and monitoring node.
Perception & Sensors	Handles local navigation and obstacle detection using the Okdo LiDAR HAT. It also manages the onboard Pi Camera V3.	Handles global perception by receiving and analyzing the RTSP video stream from the fixed TP-Link Tapo C210 camera.
Artificial Intelligence	Offloads heavy visual processing to save power and thermal load.	Runs the heavy custom CNN-based computer vision pipelines to identify fire sources and detect humans actively in danger.
Spatial Calculation	Runs the ROS 2 Navigation Stack (Nav2) and processes LiDAR data to generate local, collision-free paths to the target.	Performs Extrinsic Transformation calculations (using OpenCV and ArUco markers) to translate the 2D fire image into 3D physical coordinates.
Actuation & Hardware	Physically controls the hardware. It sends PWM signals to the BTS7960 motor drivers, controls pan-tilt servos, and triggers the 24V water pump via a relay.	Has no direct control over the robot's physical movement or suppression hardware; its role is purely computational.

Task / Feature	Raspberry Pi 4 Model B (Mobile Edge Node)	External Laptop (Vision Node)
MQTT Communication	Acts as the Subscriber . It listens to the local MQTT broker for incoming fire coordinates.	Acts as the Publisher . Once it calculates where the fire is, it sends that JSON coordinate payload to the local MQTT broker.

5. Computing and Control Hardware

5.1. Raspberry Pi 4 Model B — Mobile Control Unit

The Raspberry Pi 4 Model B serves as the robot’s onboard controller, responsible for:

- Maintaining Wi-Fi connectivity
- Subscribing to MQTT topics
- Processing LiDAR data
- Generating PWM signals for motor drivers
- Controlling servos and relays
- Executing navigation and suppression logic

Its quad-core Cortex-A76 CPU and 8 GB RAM provide ample performance for real-time robotics tasks.

5.2. Laptop — Vision Processing Unit

The external laptop handles:

- Custom CNN-based fire and human detection.
- Frame-by-frame analysis of the Tapo C210 RTSP stream.
- Coordinate extraction and transformation.
- Publishing fire-location data to the MQTT broker

Offloading vision processing significantly reduces power consumption and thermal load on the robot.

6. Actuators and Drive Electronics

6.1. Locomotion System

- **Motors:** Four JGB37-545 high-torque DC geared motors.
- **Drivers:** Dual BTS7960 H-Bridge modules
- **Voltage:** Powered directly from the 22.2 V main bus
- **Control:** PWM signals from Raspberry Pi GPIO pins

The BTS7960 drivers support up to 43 A peak current, making them ideal for high-load robotics applications.

6.2. Steering System

- **Mechanism:** 4-Wheel Skid-Steer
- **Actuator:** Driven entirely by the main JGB37-545 locomotion motors.
- **Function:** Directional control is achieved by varying the speed and rotational direction of the left and right motor banks independently, generating the lateral friction required to turn the chassis.

6.3. Fire Suppression System

- **Pump:** 24 V diaphragm pump (4 L/min)
- **Aiming:** Pan-tilt mechanism using MG996R metal-gear servos
- **Trigger:** Solenoid valve controlled via opto-isolated relay

This system enables accurate targeting and controlled water discharge.

7. Sensors and Perception Layer

7.1. Global Detection — TP-Link Tapo C210

- Provides 2K video stream via RTSP.
- Mounted in a fixed position for full-room coverage.

- Acts as the “global eye” for fire detection

7.2. Local Navigation — Okdo LiDAR HAT

- DTOF-based 360° scanning.
- Mounted directly on the Raspberry Pi GPIO header
- Provides real-time obstacle detection and mapping.
- Enables safe navigation toward target coordinates.

8. Communication and Logic Flow

8.1. MQTT Workflow

1. **Detection:** Laptop detects fire and evaluates human presence using custom CNNs.
2. **Localization:** Coordinates are computed
3. **Publishing:** Laptop publishes JSON payload to local broker
4. **Subscribing:** Raspberry Pi receives the message
5. **Execution:** Robot navigates and activates suppression system

This asynchronous communication ensures low latency and high reliability.

9. Interface and Wiring Logic

This section describes the detailed wiring strategy used to interface the Raspberry Pi 4 Model B with the motor drivers, LiDAR module, servos, pump, and solenoid valve. The design prioritizes electrical safety, logic-level compatibility, and reliable high-current operation.

9.1. Raspberry Pi 4 Model B GPIO Allocation

The Raspberry Pi 4 Model B serves as the central control unit for all low-voltage logic signals. To ensure electrical isolation from motor noise, the Pi is powered independently via an external USB-C power bank. **Note:** A shared ground line between the Pi and the main motor breadboard is strictly required to ensure stable PWM signal reference.

GPIO Pin Mapping Table

Component	Pin Name	Connection Point	Physical Pin #	Logic / Note
Pi Power (Main)	USB-C Port	External Power Bank	N/A	Dedicated Logic Power (Isolated)
	5V	DISCONNECTED	Pin 2	Do not connect to breadboard 5V rail!
	GND	Breadboard GND Rail	Pin 6	CRITICAL: Common Ground Reference
	3.3V	Breadboard 3.3V Rail	Pin 1	Used for Enable Pins
Left Motor Driver (BTS7960)	VCC	Breadboard 5V Rail (XL4015)		Logic Power
	GND	Breadboard GND Rail		Logic Ground
	R_EN	3.3V Rail		Hard-wired High (Always Enabled)

Component	Pin Name	Connection Point	Physical Pin #	Logic / Note
	L_EN	3.3V Rail		Hard-wired High (Always Enabled)
	R_PWM	GPIO 17	Pin 11	Forward PWM
	L_PWM	GPIO 27	Pin 13	Reverse PWM
Right Motor Driver (BTS7960)	VCC	Breadboard 5V Rail (XL4015)		Logic Power
	GND	Breadboard GND Rail		Logic Ground
	R_EN	3.3V Rail		Hard-wired High
	L_EN	3.3V Rail		Hard-wired High
	R_PWM	GPIO 25	Pin 22	Forward PWM
	L_PWM	GPIO 23	Pin 16	Reverse PWM

Component	Pin Name	Connection Point	Physical Pin #	Logic / Note
LIDAR HAT	5V Power (blue)		Pin 4	Draws 5V directly from Pi
	GND (red)		Pin 34	
	TX (white)		Pin 10	
	PWM (yellow)		Pin 8	
Pump Relay Module	PUMP_CONTROL	GPIO 26	Pin 37	3.3V Logic Trigger to switch 24V pump

9.2. Logic-Level Compatibility and Protection

The Raspberry Pi 4 Model B operates strictly at **3.3 V logic**, requiring careful interfacing with higher-voltage components.

9.2.1. Motor Driver Enable Logic

The BTS7960 motor drivers accept 3.3 V as a valid HIGH signal. To simplify control and reduce GPIO usage:

- **R_EN and L_EN pins are hard-wired to the Pi's 3.3 V rail.**
- This keeps both H-bridge channels permanently enabled.
- The Pi only needs to send PWM signals to control direction and speed.

This approach reduces software complexity and ensures the drivers are always ready to receive commands.

9.2.2. PWM Control Signals

The Raspberry Pi directly drives the BTS7960 PWM pins:

- **R_PWM** → Forward rotation
- **L_PWM** → Reverse rotation

The BTS7960's internal opto-isolation and logic circuitry safely accept 3.3 V PWM signals without level shifting.

9.2.3. Relay Control for Pump and Solenoid

The water pump and solenoid valve are inductive loads. To protect the Raspberry Pi:

- Both are controlled using **opto-isolated relay modules**.
- The relay input side uses 3.3 V logic from the Pi.
- The output side switches the **22.2 V Main Bus** to the pump.
- Flyback protection is handled inside the relay module.

9.2.4. LiDAR HAT Interface

The Okdo LiDAR HAT mounts directly onto the Raspberry Pi's 40-pin header.

- Uses **UART** communication.
- Draws power from the Pi's 5 V and 3.3 V rails.
- No external wiring required.
- Eliminates cable clutter and reduces signal noise.

This direct-mount strategy ensures stable communication and simplifies assembly.

9.3. Grounding Strategy

A unified grounding scheme is used across the entire robot:

- All converters, drivers, relays, and Raspberry Pi share a **common ground**.
- This prevents floating reference levels.
- Ensures accurate PWM signaling.
- Reduces electrical noise from motors and pump.

Proper grounding is essential for stable operation of mixed-voltage systems.

9.4. High-Current Wiring Considerations

Because the robot uses high-current motors and a **24 V pump**:

- Thick-gauge wires are used for the 22.2 V main bus.
- Signal wires are routed separately from power wires to avoid interference.
- The pump is wired to the main bus closest to the battery source to minimize voltage drop affecting the logic converter.

10. Kinematic and Transformation Modeling

The physical platform utilizes a 4-wheel drive system actuated by two BTS7960 motor drivers. The left motors (front and rear) are driven in parallel, as are the right motors. Consequently, the mechanical motion of the platform is governed by the **Skid-Steer Kinematic Model**.

Unlike standard differential drive systems, turning a skid-steer chassis requires overcoming lateral friction, causing the wheels to slip across the ground.

10.1. System Parameters

Based on the physical dimensions established in the robot's unified robot description format (URDF):

- **Track Width (B):** The lateral distance between the centerlines of the left and right wheels is 0.35 meters (calculated from the y -axis offsets of 0.175 and -0.175).
- **Wheelbase (L):** The longitudinal distance between the front and rear wheels is 0.40 meters (calculated from the x -axis offsets of 0.2 and -0.2).
- **Wheel Radius (R):** 0.065 meters.

10.2. Inverse Kinematics (Velocity Commands to Wheel Speeds)

Inverse kinematics determines the necessary rotational velocities of the left and right wheel sets (v_L) and (v_R) to achieve a desired linear velocity (v) and angular velocity (ω) for the robot chassis.

Assuming ideal conditions where the front and rear wheels on each side rotate at the identical speed ($v_{front_left} = v_{rear_left} = v_L$), the equations are:

$$v_L = v - \frac{\omega \cdot B}{2}$$

$$v_R = v + \frac{\omega \cdot B}{2}$$

To convert these linear tangential wheel speeds to the angular velocity of the motors (ω_m , measured in radians per second):

$$\omega_{L_motor} = \frac{v_L}{R}$$

$$\omega_{R_motor} = \frac{v_R}{R}$$

10.3. Forward Kinematics (Wheel Speeds to Robot Motion)

Forward kinematics calculates the robot's resulting linear and angular velocities (v and ω) in the world frame based on the current velocities of the left and right wheel sets (v_L and v_R).

To accurately model a skid-steer robot in the real world, an empirical **slip factor (c)** is introduced to account for the efficiency lost to lateral friction during a turn.

$$v = \frac{v_R + v_L}{2}$$

$$\omega = \frac{v_R - v_L}{B \cdot c}$$

The robot's pose (x, y, θ) is estimated by integrating these velocities over time:

$$x(t) = \int v \cdot \cos(\theta) dt$$

$$y(t) = \int v \cdot \sin(\theta) dt$$

$$\theta(t) = \int \omega dt$$

10.4. Homogeneous Transformation Matrices (TF)

Transformation matrices define the precise spatial relationships (translation and rotation) between the center of the robot (base_link) and its various sensors and actuators. These are modeled as 4×4 homogeneous matrices (T):

$$T = \begin{bmatrix} R_{3 \times 3} & P_{3 \times 1} \\ 0 & 1 \end{bmatrix}$$

- **Base to Okdo LiDAR**

The lidar_joint connects the lidar_link to the base_link with a pure translation of 0.26 meters along the z-axis and zero rotation.

$$T_{base}^{lidar} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.26 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Base to Pi Camera V3**

The camera_joint connects the camera_link to the base_link with a translation of 0.25 meters on the x-axis, 0.125 meters on the z-axis, and zero rotation.

$$T_{base}^{camera} = \begin{bmatrix} 1 & 0 & 0 & 0.25 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.125 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Base to Front-Left Wheel**

The front_left_wheel_joint involves both translation and rotation. It is translated 0.2 meters (x) and 0.175 meters (y) from the base. To align the wheel cylinder properly, it is rotated -1.5708 radians (-90°) around the x -axis (Roll).

Using the standard rotation matrix for the x -axis, where $\cos(-90^\circ) = \text{and } \sin(-90^\circ) = -1$:

$$T_{base}^{FL_wheel} = \begin{bmatrix} 1 & 0 & 0 & 0.2 \\ 0 & 0 & 1 & 0.175 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

10.5. Extrinsic Transformation Modeling (Surveillance Camera to Robot Frame)

To achieve full autonomy, the system utilizes an external surveillance camera running a custom CNN vision processing node to detect flames and prioritize human safety. To bridge

the camera's fixed coordinate frame with the mobile robot's coordinate frame, an extrinsic transformation matrix (T_{camera}^{robot}) must be computed dynamically in real-time.

This mapping is achieved using fiducial markers (specifically, ArUco markers) mounted securely on the robot chassis. By detecting the known geometric properties of the marker, the vision system solves the Perspective-n-Point (PnP) problem to extract the physical translation vector ($tvec$) and rotation vector ($rvec$) of the robot relative to the camera lens.

The rotation vector is converted into a 3×3 rotation matrix (R) using Rodrigues' rotation formula. The resulting components are combined to construct the 4×4 homogeneous transformation matrix:

$$T_{camera}^{robot} = \begin{bmatrix} R_{11} & R_{12} & R_{13} & tvec_x \\ R_{21} & R_{22} & R_{23} & tvec_y \\ R_{31} & R_{32} & R_{33} & tvec_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Calculating this matrix allows the system to map the 2D pixel coordinates of the detected flame into the physical 3D workspace. This spatial data is then transmitted to the robot via the local MQTT broker, enabling the ROS 2 Navigation Stack to compute a precise path to the target.

10.5.1. Vision Node Implementation (OpenCV, PyTorch & Flask)

The following script runs on the external Vision Node (laptop). It utilizes multithreading to capture RTSP streams from the Tapo C210 and Raspberry Pi cameras without blocking. It simultaneously calculates the robot's real-time coordinate transformation matrix via ArUco markers, runs a custom PyTorch MiniYOLO model for global fire detection, and executes Keras models for local human-in-danger prioritization. A Flask server is also initialized to broadcast the processed video feeds.

Python

```
import os
import cv2
import numpy as np
import torch
import torch.nn as nn
from dotenv import load_dotenv
import tensorflow as tf
import threading
import time
```

```

from flask import Flask, Response
from flask_cors import CORS

# Load environment variables from a .env file if present
load_dotenv()

app = Flask(__name__)
CORS(app) # Allow CORS so the website can fetch the streams

latest_frame_tapo = None
latest_frame_pi = None

def generate_frames(camera_type):
    global latest_frame_tapo, latest_frame_pi
    while True:
        frame = None
        if camera_type == 'tapo':
            frame = latest_frame_tapo
        elif camera_type == 'pi':
            frame = latest_frame_pi

        if frame is None:
            time.sleep(0.01)
            continue

        yield (b'--frame\r\n'
              b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')

@app.route('/video_feed_tapo')
def video_feed_tapo():
    return Response(generate_frames('tapo'), mimetype='multipart/x-mixed-replace;
boundary=frame')

@app.route('/video_feed_pi')
def video_feed_pi():
    return Response(generate_frames('pi'), mimetype='multipart/x-mixed-replace;
boundary=frame')

def run_flask():
    app.run(host='0.0.0.0', port=5000, debug=False, use_reloader=False)

class MiniYOLO(nn.Module):
    def __init__(self, S=7, C=2):
        super().__init__()

```

```

self.S = S
self.C = C

self.backbone = nn.Sequential(
    nn.Conv2d(3,16,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(16,32,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(32,64,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(64,128,3,padding=1), nn.ReLU(),
    nn.AdaptiveAvgPool2d((7,7))
)

self.head = nn.Conv2d(128, 5+2, kernel_size=1)

def forward(self, x):
    x = self.backbone(x)
    x = self.head(x)
    x = x.permute(0,2,3,1)
    return x

def calculate_transformation_matrix(image_frame, camera_matrix, dist_coeffs,
marker_length=0.10):
    """
    Detects an ArUco marker and calculates the 4x4 transformation matrix.
    marker_length is in meters (e.g., 0.10 = 10cm).
    """
    # Load the ArUco dictionary (using 4x4 dictionary for standard markers)
    aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
    parameters = cv2.aruco.DetectorParameters()
    detector = cv2.aruco.ArucoDetector(aruco_dict, parameters)

    # Detect the markers in the image
    corners, ids, rejected = detector.detectMarkers(image_frame)

    transformation_matrix = None

    if ids is not None:
        # Define the 3D coordinates of the marker corners in its own coordinate
system
        obj_points = np.array([
            [-marker_length/2, marker_length/2, 0],
            [marker_length/2, marker_length/2, 0],
            [marker_length/2, -marker_length/2, 0],
            [-marker_length/2, -marker_length/2, 0]
        ], dtype=np.float32)

```

```

        # Iterate through detected markers (assuming ID 0 is the robot)
        for i in range(len(ids)):
            # Solve PnP to get the rotation and translation vectors
            success, rvec, tvec = cv2.solvePnP(
                obj_points, corners[i], camera_matrix, dist_coeffs,
                flags=cv2.SOLVEPNP_IPPE_SQUARE
            )

            if success:
                # Convert the 3x1 rotation vector (rvec) to a 3x3 rotation matrix
                # using Rodrigues
                rotation_matrix, _ = cv2.Rodrigues(rvec)

                # Initialize a 4x4 identity matrix
                transformation_matrix = np.eye(4, dtype=np.float32)

                # Plug in the 3x3 rotation matrix
                transformation_matrix[0:3, 0:3] = rotation_matrix

                # Plug in the 3x1 translation vector (tvec)
                transformation_matrix[0:3, 3] = tvec.flatten()

                # Draw axes on the marker for visual debugging
                cv2.drawFrameAxes(image_frame, camera_matrix, dist_coeffs, rvec,
                                tvec, 0.05)

                break # Assuming we only track one robot marker

        return transformation_matrix, image_frame

class CameraStream:
    """
    Continually grabs frames from the camera in a background thread.
    This prevents the OpenCV buffer from filling up and causing massive delays
    when running heavy neural networks (like PyTorch and Keras).
    """
    def __init__(self, src):
        self.stream = cv2.VideoCapture(src)
        self.stream.set(cv2.CAP_PROP_BUFFERSIZE, 1)
        self.grabbed, self.frame = self.stream.read()
        self.stopped = False

    def start(self):
        if self.stream.isOpened():
            self.thread = threading.Thread(target=self.update, args=())
            self.thread.daemon = True

```

```

        self.thread.start()
    return self

def update(self):
    while not self.stopped:
        if not self.stream.isOpened():
            self.stopped = True
            break
        # Drains the internal buffer constantly, keeping only the most recent
frame
        self.grabbed, self.frame = self.stream.read()

def read(self):
    return self.grabbed, self.frame

def stop(self):
    self.stopped = True
    if hasattr(self, 'thread'):
        self.thread.join()
    self.stream.release()

def isOpened(self):
    return self.stream.isOpened()

if __name__ == '__main__':
    # 1. Load the Custom Fire Detection Model (MiniYOLO)
    print("Loading Custom Fire Detection model...")
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = MiniYOLO(S=7, C=2).to(device)

    # Construct the absolute path to the model file to avoid FileNotFoundError
    current_dir = os.path.dirname(os.path.abspath(__file__))
    model_path = os.path.join(current_dir, "Fire_Detection_TapoC210",
"fire_detection_model.pt")

    # Load the state dict.
    model.load_state_dict(torch.load(model_path, map_location=device))
    model.eval()

    # Start Flask video streaming server in a background thread
    print("Starting Flask streaming server on port 5000...")
    print("\n" + "="*60)
    print("🌐 WEBSITE CAMERA URLs:")
    print("➔ Tapo Camera URL: http://127.0.0.1:5000/video_feed_tapo")
    print("➔ Pi Camera URL: http://127.0.0.1:5000/video_feed_pi")

```

```

print("="*60 + "\n")
flask_thread = threading.Thread(target=run_flask)
flask_thread.daemon = True
flask_thread.start()

# 1.1 Load the Keras Models for Raspberry Pi (Fire and Human)
print("Loading Raspberry Pi Fire and Human Detection models...")
fire_model_pi_path = os.path.join(current_dir, "Fire_Detection_Raspberry_Pi",
"fire_robust_model.h5")
human_model_pi_path = os.path.join(current_dir, "Human_Detection",
"human.h5")

try:
    fire_model_pi = tf.keras.models.load_model(fire_model_pi_path)
    human_model_pi = tf.keras.models.load_model(human_model_pi_path)
    print("Raspberry Pi models loaded successfully.")
except Exception as e:
    print(f"Error loading Keras models: {e}")
    fire_model_pi, human_model_pi = None, None

# Initialize cameras
# Provide the RTSP URL via environment variable or place it directly below
tapo_rtsp_url = os.environ.get("RTSP_URL")
cap_tapo = CameraStream(tapo_rtsp_url).start()

if not cap_tapo.isOpened():
    print(f"Failed to open video stream. Input used: {tapo_rtsp_url}")
    exit(1)

# Initialize Raspberry Pi Camera Module 3 stream
# Since this runs on the laptop, the Pi must stream its camera over the
network (e.g., via RTSP, HTTP, UDP, TCP).
# To stream with MAX FOV from a Pi Camera Module 3 (avoiding center-crop),
run this command on the Raspberry Pi:
# sudo rpivid -n -t 0 --mode 4608:2592:12 --width 640 --height 360 --
framerate 30 --codec mjpeg --listen -o tcp://0.0.0.0:8888
# Then in your laptop's .env file set: PI_CAMERA_URL="tcp://<PI_IP>:8888"
pi_camera_url = os.environ.get("PI_CAMERA_URL")
if pi_camera_url:
    cap_pi = CameraStream(pi_camera_url).start()
else:
    print("PI_CAMERA_URL not set in .env. Falling back to the laptop webcam
(0) for testing the Pi models.")
    cap_pi = CameraStream(0).start()

```

```

    if not cap_pi.isOpened():
        print("Failed to open Raspberry Pi Camera stream. Models will skip Pi
frames.")

    # Placeholder camera matrix
    placeholder_camera_matrix = np.array([[800, 0, 320], [0, 800, 240], [0, 0,
1]], dtype=np.float32)
    placeholder_dist_coeffs = np.zeros((4,1))

    print("Starting Robot Tracking and Fire Detection...")

    while True:
        # Get latest frames from background threads completely instantly and
without blocking
        ret_tapo, frame_tapo = cap_tapo.read()
        ret_pi, frame_pi = False, None

        if cap_pi.isOpened():
            ret_pi, frame_pi = cap_pi.read()

        if not ret_tapo or frame_tapo is None:
            time.sleep(0.01) # Avoid 100% CPU lock while stream connects or
reconnects
            continue

        # 2. Run ArUco tracking for the robot (Kinematics) on Tapo
        T_matrix, display_frame_tapo = calculate_transformation_matrix(
            frame_tapo,
            placeholder_camera_matrix,
            placeholder_dist_coeffs,
            marker_length=0.093
        )

        if T_matrix is not None:
            # We found the robot!
            robot_x = T_matrix[0, 3]
            robot_y = T_matrix[1, 3]
            # print(f"Robot Location: X:{robot_x:.2f}, Y:{robot_y:.2f}")

        # 3. Run Custom Fire Detection on the SAME Tapo frame
        # Preprocess
        img_rgb = cv2.cvtColor(display_frame_tapo, cv2.COLOR_BGR2RGB)
        h_orig, w_orig = display_frame_tapo.shape[:2]

        # Resize to 416x416, Swap axes (HWC to CHW), and normalize (0-1)

```

```

img_input = cv2.resize(img_rgb, (416, 416)).transpose(2, 0, 1) / 255.0
tensor = torch.tensor(img_input,
dtype=torch.float32).unsqueeze(0).to(device)

with torch.no_grad():
    out = model(tensor) # Shape: (1, 7, 7, 7)

# Parse output for the single highest confidence prediction
S = 7
conf_map = out[0, ..., 4]
flat_idx = torch.argmax(conf_map)
j, i = torch.unravel_index(flat_idx, (S, S))

conf = conf_map[j, i].item()

fire_active = False # Flag you can use to trigger MQTT

if conf > 0.50: # Only show detections with > 50% confidence
    # Get prediction data
    box = out[0, j, i, 0:4] # x_cell, y_cell, w, h
    cls = torch.argmax(out[0, j, i, 5:]).item()

    # Convert back to original pixel coordinates
    x_abs = ((box[0] + i) / S * w_orig).item()
    y_abs = ((box[1] + j) / S * h_orig).item()
    w_abs = (box[2] * w_orig).item()
    h_abs = (box[3] * h_orig).item()

    # Bounding box corners
    x1 = int(x_abs - w_abs / 2)
    y1 = int(y_abs - h_abs / 2)
    x2 = int(x_abs + w_abs / 2)
    y2 = int(y_abs + h_abs / 2)

    # Checking if the class detected represents fire (assuming 0 or 1 is
fire dependening on the labels)
    if cls == 0 or cls == 1:
        fire_active = True

    # Draw a red bounding box around the fire
    cv2.rectangle(display_frame_tapo, (x1, y1), (x2, y2), (0, 0,
255), 3)

    # Add label and confidence score
    label = f"Fire: {conf:.2f}"

```

```

        cv2.putText(display_frame_tapo, label, (x1, y1 - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)

# 4. Run Raspberry Pi Fire & Human Detection (Keras) on the Pi Camera
frame
display_frame_pi = None
if ret_pi and frame_pi is not None and fire_model_pi is not None and
human_model_pi is not None:
    display_frame_pi = frame_pi.copy()
    img_size = 128

    # Preprocess for Fire model (requires manual scaling)
    fire_pi_img = cv2.resize(frame_pi, (img_size, img_size))
    fire_pi_img = cv2.cvtColor(fire_pi_img, cv2.COLOR_BGR2RGB)
    fire_pi_img = fire_pi_img.astype(np.float32) / 255.0
    fire_pi_input = np.expand_dims(fire_pi_img, axis=0)

    fire_pi_pred = fire_model_pi.predict(fire_pi_input, verbose=0)[0][0]

    # Preprocess for Human model (rescaling layer is inside the model)
    human_pi_img = cv2.resize(frame_pi, (img_size, img_size))
    human_pi_img = cv2.cvtColor(human_pi_img, cv2.COLOR_BGR2RGB)
    human_pi_img = human_pi_img.astype(np.float32)
    human_pi_input = np.expand_dims(human_pi_img, axis=0)

    human_pi_pred = human_model_pi.predict(human_pi_input,
verbose=0)[0][0]

    # Draw labels for Keras models
    font = cv2.FONT_HERSHEY_SIMPLEX
    cv2.putText(display_frame_pi, f"Pi Fire: {fire_pi_pred:.2f}", (10,
30), font, 0.8, (0, 0, 255) if fire_pi_pred > 0.5 else (0, 255, 0), 2)
    cv2.putText(display_frame_pi, f"Pi Human: {human_pi_pred:.2f}", (10,
60), font, 0.8, (255, 0, 0) if human_pi_pred > 0.5 else (0, 255, 0), 2)

# Update global frames for Flask stream
if display_frame_tapo is not None:
    ret, buffer = cv2.imencode('.jpg', display_frame_tapo)
    if ret:
        latest_frame_tapo = buffer.tobytes()
if display_frame_pi is not None:
    ret, buffer = cv2.imencode('.jpg', display_frame_pi)
    if ret:
        latest_frame_pi = buffer.tobytes()

```

```

# --- Display the Windows ---
cv2.namedWindow('Vision Node: Tapo Tracking & AI', cv2.WINDOW_NORMAL)
cv2.resizeWindow('Vision Node: Tapo Tracking & AI', 1280, 720)
cv2.imshow('Vision Node: Tapo Tracking & AI', display_frame_tapo)

if display_frame_pi is not None:
    cv2.namedWindow('Vision Node: Pi Camera AI', cv2.WINDOW_NORMAL)
    cv2.resizeWindow('Vision Node: Pi Camera AI', 640, 480)
    cv2.imshow('Vision Node: Pi Camera AI', display_frame_pi)

# Press 'q' to quit
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap_tapo.stop()
if cap_pi.isOpened():
    cap_pi.stop()
cv2.destroyAllWindows()

```

11. ROS 2 System Integration & Control

11.1. Phase 1: Simulation and Robot Modeling (Gazebo Harmonic & WSL Environment)

The first phase involves creating the physical blueprint of the robot (URDF) and deploying it into a physics-enabled simulation environment using Gazebo Harmonic. Because the development environment operates within Windows Subsystem for Linux (WSL), specific rendering engines (ogre) and resolution constraints were applied to ensure stability.

Step 1: The URDF File (phoenix.urdf)

Path: ~/ambers_ws/src/phoenix_description/urdf/phoenix.urdf

This XML file describes the 4-wheel skid-steer chassis, applying precise collision boundaries, mass, and inertial matrices. It also embeds Gazebo Harmonic plugins to simulate the Okdo LiDAR, Raspberry Pi Camera V3, and the skid-steer drivetrain. A dummy base_footprint link is utilized to satisfy ROS 2 KDL parsing requirements.

```

<?xml version="1.0"?>
<robot name="phoenix">

  <link name="base_footprint"/>

```

```

<joint name="base_footprint_joint" type="fixed">
  <parent link="base_footprint"/>
  <child link="base_link"/>
  <origin xyz="0 0 0" rpy="0 0 0"/>
</joint>

<material name="black">
  <color rgba="0.1 0.1 0.1 1.0"/>
</material>
<material name="silver">
  <color rgba="0.75 0.75 0.75 1.0"/>
</material>
<material name="blue">
  <color rgba="0.0 0.0 0.8 1.0"/>
</material>
<material name="red">
  <color rgba="1.0 0.0 0.0 1.0"/>
</material>

<link name="base_link">
  <visual>
    <origin xyz="0 0 0.125" rpy="0 0 0"/>
    <geometry>
      <box size="0.5 0.3 0.25"/>
    </geometry>
    <material name="red"/>
  </visual>
  <collision>
    <origin xyz="0 0 0.125" rpy="0 0 0"/>
    <geometry>
      <box size="0.5 0.3 0.25"/>
    </geometry>
  </collision>
  <inertial>
    <origin xyz="0 0 0.125" rpy="0 0 0"/>
    <mass value="5.0"/>
    <inertia ixx="0.0635" ixy="0" ixz="0" iyy="0.1302" iyz="0" izz="0.1416"/>
  </inertial>
</link>
<gazebo reference="base_link">
  <material>Gazebo/Red</material>
</gazebo>

<link name="front_left_wheel">

```

```

<visual>
  <geometry>
    <cylinder radius="0.065" length="0.05"/>
  </geometry>
  <material name="blue"/>
</visual>
<collision>
  <geometry>
    <cylinder radius="0.065" length="0.05"/>
  </geometry>
</collision>
<inertial>
  <mass value="0.2"/>
  <inertia ixx="0.00025" ixy="0" ixz="0" iyy="0.00025" iyz="0"
izz="0.00042"/>
</inertial>
</link>
<joint name="front_left_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="front_left_wheel"/>
  <origin xyz="0.2 0.175 0" rpy="-1.5708 0 0"/>
  <axis xyz="0 0 1"/>
</joint>
<gazebo reference="front_left_wheel">
  <material>Gazebo/Blue</material>
  <mu1>1.0</mu1>
  <mu2>0.05</mu2>
  <fdir1>1 0 0</fdir1>
</gazebo>

<link name="front_right_wheel">
  <visual>
    <geometry>
      <cylinder radius="0.065" length="0.05"/>
    </geometry>
    <material name="blue"/>
  </visual>
  <collision>
    <geometry>
      <cylinder radius="0.065" length="0.05"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="0.2"/>

```

```

    <inertia ixx="0.00025" ixy="0" ixz="0" iyy="0.00025" iyz="0"
izz="0.00042"/>
  </inertial>
</link>
<joint name="front_right_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="front_right_wheel"/>
  <origin xyz="0.2 -0.175 0" rpy="-1.5708 0 0"/>
  <axis xyz="0 0 1"/>
</joint>
<gazebo reference="front_right_wheel">
  <material>Gazebo/Blue</material>
  <mu1>1.0</mu1>
  <mu2>0.05</mu2>
  <fdir1>1 0 0</fdir1>
</gazebo>

<link name="rear_left_wheel">
  <visual>
    <geometry>
      <cylinder radius="0.065" length="0.05"/>
    </geometry>
    <material name="silver"/>
  </visual>
  <collision>
    <geometry>
      <cylinder radius="0.065" length="0.05"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="0.2"/>
    <inertia ixx="0.00025" ixy="0" ixz="0" iyy="0.00025" iyz="0"
izz="0.00042"/>
  </inertial>
</link>
<joint name="rear_left_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="rear_left_wheel"/>
  <origin xyz="-0.2 0.175 0" rpy="-1.5708 0 0"/>
  <axis xyz="0 0 1"/>
</joint>
<gazebo reference="rear_left_wheel">
  <material>Gazebo/Silver</material>
  <mu1>1.0</mu1>
  <mu2>0.05</mu2>

```

```

    <fdir1>1 0 0</fdir1>
</gazebo>

<link name="rear_right_wheel">
  <visual>
    <geometry>
      <cylinder radius="0.065" length="0.05"/>
    </geometry>
    <material name="silver"/>
  </visual>
  <collision>
    <geometry>
      <cylinder radius="0.065" length="0.05"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="0.2"/>
    <inertia ixx="0.00025" ixy="0" ixz="0" iyy="0.00025" iyz="0"
izz="0.00042"/>
  </inertial>
</link>
<joint name="rear_right_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="rear_right_wheel"/>
  <origin xyz="-0.2 -0.175 0" rpy="-1.5708 0 0"/>
  <axis xyz="0 0 1"/>
</joint>
<gazebo reference="rear_right_wheel">
  <material>Gazebo/Silver</material>
  <mu1>1.0</mu1>
  <mu2>0.05</mu2>
  <fdir1>1 0 0</fdir1>
</gazebo>

<link name="lidar_link">
  <visual>
    <geometry>
      <cylinder radius="0.035" length="0.02"/>
    </geometry>
    <material name="black"/>
  </visual>
  <collision>
    <geometry>
      <cylinder radius="0.035" length="0.02"/>
    </geometry>
  </collision>

```

```

    </collision>
    <inertial>
      <mass value="0.1"/>
      <inertia ixx="0.000034" ixy="0" ixz="0" iyy="0.000034" iyz="0"
izz="0.000061"/>
    </inertial>
  </link>
  <joint name="lidar_joint" type="fixed">
    <parent link="base_link"/>
    <child link="lidar_link"/>
    <origin xyz="0 0 0.26" rpy="0 0 0"/>
  </joint>
  <gazebo reference="lidar_link">
    <material>Gazebo/Black</material>
  </gazebo>

  <link name="camera_link">
    <visual>
      <geometry>
        <box size="0.025 0.024 0.0115"/>
      </geometry>
      <material name="silver"/>
    </visual>
    <collision>
      <geometry>
        <box size="0.025 0.024 0.0115"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="0.01"/>
      <inertia ixx="0.000001" ixy="0" ixz="0" iyy="0.000001" iyz="0"
izz="0.000001"/>
    </inertial>
  </link>
  <joint name="camera_joint" type="fixed">
    <parent link="base_link"/>
    <child link="camera_link"/>
    <origin xyz="0.27 0 0.125" rpy="0 0 0"/> </joint>

  <gazebo>
    <plugin filename="gz-sim-diff-drive-system"
name="gz::sim::systems::DiffDrive">
      <left_joint>front_left_wheel_joint</left_joint>
      <left_joint>rear_left_wheel_joint</left_joint>
      <right_joint>front_right_wheel_joint</right_joint>

```

```

    <right_joint>rear_right_wheel_joint</right_joint>
    <wheel_separation>0.35</wheel_separation>
    <wheel_radius>0.065</wheel_radius>
    <odom_publish_frequency>30</odom_publish_frequency>
    <topic>/cmd_vel</topic>

    <frame_id>odom</frame_id>
    <child_frame_id>base_footprint</child_frame_id>
  </plugin>
</gazebo>

<gazebo reference="lidar_link">
  <sensor type="gpu_lidar" name="lidar_sensor">
    <update_rate>10.0</update_rate>
    <topic>scan</topic>
    <gz_frame_id>lidar_link</gz_frame_id>
    <ray>
      <scan>
        <horizontal>
          <samples>360</samples>
          <resolution>1</resolution>
          <min_angle>0.0</min_angle>
          <max_angle>6.28319</max_angle>
        </horizontal>
      </scan>
      <range>
        <min>0.35</min>
        <max>10.0</max>
      </range>
    </ray>
    <always_on>true</always_on>
    <visualize>true</visualize>
  </sensor>
</gazebo>

<gazebo reference="camera_link">
  <sensor type="camera" name="camera_sensor">
    <update_rate>30.0</update_rate>
    <topic>phoenix/camera/image_raw</topic>
    <gz_frame_id>camera_link</gz_frame_id>
    <camera>
      <horizontal_fov>1.309</horizontal_fov>
      <image>
        <width>640</width>    <height>480</height> <format>R8G8B8</format>
      </image>
    </camera>
  </sensor>
</gazebo>

```

```

    <clip>
      <near>0.02</near>
      <far>300</far>
    </clip>
  </camera>
  <always_on>true</always_on>
  <visualize>true</visualize>
</sensor>
</gazebo>

</robot>

```

Note: To ensure seamless transition between simulation and hardware, the bridge maintains a direct 1:1 mapping for the /cmd_vel topic. This allows the same ROS 2 navigation signals to drive the virtual Gazebo model and the physical motor controller node without remapping

Step 2: The Custom Simulation World (warehouse.sdf)

Path: ~/ambers_ws/src/phoenix_description/worlds/warehouse.sdf To validate the SLAM and autonomous navigation pipelines, a custom 20x20m Gazebo world was constructed featuring industrial shelving aisles and a permanent target (a 1-meter red box) positioned in the back quadrant. Proper <diffuse> and <ambient> lighting parameters were added to the environment materials to ensure compatibility with the Ogre rendering engine used in WSL.

```

<?xml version="1.0" ?>
<sdf version="1.8">
  <world name="warehouse">
    <physics name="1ms" type="ignored">
      <max_step_size>0.001</max_step_size>
      <real_time_factor>1.0</real_time_factor>
    </physics>
    <plugin filename="gz-sim-physics-system"
name="gz::sim::systems::Physics"></plugin>
    <plugin filename="gz-sim-user-commands-system"
name="gz::sim::systems::UserCommands"></plugin>
    <plugin filename="gz-sim-scene-broadcaster-system"
name="gz::sim::systems::SceneBroadcaster"></plugin>

    <plugin filename="gz-sim-sensors-system" name="gz::sim::systems::Sensors">
      <render_engine>ogre</render_engine>
    </plugin>

    <light type="directional" name="sun">

```

```

    <cast_shadows>true</cast_shadows>
    <pose>0 0 10 0 0 0</pose>
    <diffuse>0.8 0.8 0.8 1</diffuse>
    <specular>0.2 0.2 0.2 1</specular>
    <direction>-0.5 0.1 -0.9</direction>
  </light>

  <model name="ground_plane">
    <static>true</static>
    <link name="link">
      <collision name="collision"><geometry><plane><normal>0 0
1</normal><size>100 100</size></plane></geometry></collision>
      <visual name="visual">
        <geometry><plane><normal>0 0 1</normal><size>100
100</size></plane></geometry>
        <material>
          <ambient>0.7 0.7 0.7 1</ambient>
          <diffuse>0.7 0.7 0.7 1</diffuse>
        </material>
      </visual>
    </link>
  </model>

  <model name="walls">
    <static>true</static>
    <link name="wall_north">
      <pose>0 10 1.5 0 0 0</pose>
      <collision name="col"><geometry><box><size>20 0.2
3</size></box></geometry></collision>
      <visual name="vis">
        <geometry><box><size>20 0.2 3</size></box></geometry>
        <material><ambient>0.6 0.7 0.8 1</ambient><diffuse>0.6 0.7 0.8
1</diffuse></material>
      </visual>
    </link>
    <link name="wall_south">
      <pose>0 -10 1.5 0 0 0</pose>
      <collision name="col"><geometry><box><size>20 0.2
3</size></box></geometry></collision>
      <visual name="vis">
        <geometry><box><size>20 0.2 3</size></box></geometry>
        <material><ambient>0.6 0.7 0.8 1</ambient><diffuse>0.6 0.7 0.8
1</diffuse></material>
      </visual>
    </link>
  </model>

```

```

    <link name="wall_east">
      <pose>10 0 1.5 0 0 0</pose>
      <collision name="col"><geometry><box><size>0.2 20
3</size></box></geometry></collision>
      <visual name="vis">
        <geometry><box><size>0.2 20 3</size></box></geometry>
        <material><ambient>0.6 0.7 0.8 1</ambient><diffuse>0.6 0.7 0.8
1</diffuse></material>
      </visual>
    </link>
    <link name="wall_west">
      <pose>-10 0 1.5 0 0 0</pose>
      <collision name="col"><geometry><box><size>0.2 20
3</size></box></geometry></collision>
      <visual name="vis">
        <geometry><box><size>0.2 20 3</size></box></geometry>
        <material><ambient>0.6 0.7 0.8 1</ambient><diffuse>0.6 0.7 0.8
1</diffuse></material>
      </visual>
    </link>
  </model>

  <model name="shelving_unit_1">
    <static>true</static>
    <pose>0 3 1 0 0 0</pose>
    <link name="link">
      <collision name="col"><geometry><box><size>12 1
2</size></box></geometry></collision>
      <visual name="vis">
        <geometry><box><size>12 1 2</size></box></geometry>
        <material>
          <ambient>0.8 0.4 0.1 1</ambient>
          <diffuse>0.8 0.4 0.1 1</diffuse>
        </material>
      </visual>
    </link>
  </model>

  <model name="shelving_unit_2">
    <static>true</static>
    <pose>0 -3 1 0 0 0</pose>
    <link name="link">
      <collision name="col"><geometry><box><size>12 1
2</size></box></geometry></collision>
      <visual name="vis">

```

```

    <geometry><box><size>12 1 2</size></box></geometry>
    <material>
      <ambient>0.8 0.4 0.1 1</ambient>
      <diffuse>0.8 0.4 0.1 1</diffuse>
    </material>
  </visual>
</link>
</model>

<model name="red_target_box">
  <pose>8 8 0.5 0 0 0</pose>
  <static>true</static>
  <link name="link">
    <collision name="collision"><geometry><box><size>1 1
1</size></box></geometry></collision>
    <visual name="visual">
      <geometry><box><size>1 1 1</size></box></geometry>
      <material>
        <ambient>1 0 0 1</ambient>
        <diffuse>1 0 0 1</diffuse>
      </material>
    </visual>
  </link>
</model>
</world>
</sdf>

```

Step 3: The Gazebo Launch and Bridge File (gazebo.launch.py)

Path: ~/ambers_ws/src/phoenix_description/launch/gazebo.launch.py

This launch file initializes the robot state publisher, a joint state publisher (to broadcast the continuous wheel joint positions), spawns the custom warehouse world, injects the URDF entity, and establishes the crucial ROS_GZ parameter bridges. To enable SLAM, Odometry (/odom) and Transform (/tf) topics generated by the Gazebo DiffDrive plugin were added to the bridge.

```

import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node

```

```

def generate_launch_description():
    pkg_name = 'phoenix_description'
    pkg_share = get_package_share_directory(pkg_name)
    urdf_file_path = os.path.join(pkg_share, 'urdf', 'phoenix.urdf')

    with open(urdf_file_path, 'r') as infp:
        robot_desc = infp.read()

    # 1. State Publisher
    robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        name='robot_state_publisher',
        output='screen',
        parameters=[{'robot_description': robot_desc, 'use_sim_time': True}]
    )

    # 1.5 Joint State Publisher (Fixes the red wheel errors)
    joint_state_publisher_node = Node(
        package='joint_state_publisher',
        executable='joint_state_publisher',
        name='joint_state_publisher',
        parameters=[{'use_sim_time': True}]
    )

    # 2. Start Gazebo Harmonic with Custom World
    ros_gz_sim_dir = get_package_share_directory('ros_gz_sim')
    world_file_path = os.path.join(pkg_share, 'worlds', 'warehouse.sdf')

    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(ros_gz_sim_dir, 'launch', 'gz_sim.launch.py')
        ),
        # Load the custom world file instead of empty.sdf
        launch_arguments={'gz_args': f'-r {world_file_path}'}.items()
    )

    # 3. Spawn Phoenix
    spawn_entity = Node(
        package='ros_gz_sim',
        executable='create',
        arguments=['-topic', 'robot_description', '-name', 'phoenix'],
        output='screen'
    )

    # 4. Bridge ROS 2 and Gazebo Harmonic Topics
    bridge = Node(

```

```

package='ros_gz_bridge',
executable='parameter_bridge',
arguments=[
    '/cmd_vel@geometry_msgs/msg/Twist[gz.msgs.Twist]',
    'scan@sensor_msgs/msg/LaserScan[gz.msgs.LaserScan]',
    'phoenix/camera/image_raw@sensor_msgs/msg/Image[gz.msgs.Image]',

    # --- ADD THESE TWO LINES FOR ODOMETRY & TF ---
    '/model/phoenix/tf@tf2_msgs/msg/TFMessage[gz.msgs.Pose_V]',
    '/model/phoenix/odometry@nav_msgs/msg/Odometry[gz.msgs.Odometry'
],
remappings=[
    ('scan', '/scan'),
    ('phoenix/camera/image_raw', '/phoenix/camera/image_raw'),

    # --- ADD THESE TWO LINES TO REMAP TO STANDARD ROS TOPICS ---
    ('/model/phoenix/tf', '/tf'),
    ('/model/phoenix/odometry', '/odom')
],
output='screen'
)
return LaunchDescription([
    robot_state_publisher_node,
    joint_state_publisher_node,
    gazebo,
    spawn_entity,
    bridge
])

```

Step 4: CMake Configuration and Building

Path: ~/ambers_ws/src/phoenix_description/CMakeLists.txt

The worlds and launch directories must be registered for ROS 2 to locate them during execution.

CMake

install(DIRECTORY

urdf

launch

worlds

```
DESTINATION share/${PROJECT_NAME}
)
```

Build Command (with symlinks for rapid development):

Bash

```
cd ~/ambers_ws
```

```
colcon build --packages-select phoenix_description --symlink-install
```

```
source install/setup.bash
```

```
ros2 launch phoenix_description gazebo.launch.py
```

11.2. Phase 2: Hardware Control (Raspberry Pi 4 Model B)

The second phase bridges ROS 2 velocity commands (/cmd_vel) to the physical 22.2V motor subsystem via the BTS7960 drivers.

Step 1: Create the Control Package

Bash

```
cd ~/ambers_ws/src
ros2 pkg create --build-type ament_python phoenix_control --dependencies rclpy
geometry_msgs
cd phoenix_control/phoenix_control
```

Step 2: Motor Controller Node (motor_controller.py)

Path: ~/ambers_ws/src/phoenix_control/phoenix_control/motor_controller.py

Python

```
# Contributor: Bassel Elbahnasy
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from gpiozero import PWMOutputDevice

class MotorController(Node):
    def __init__(self):
        super().__init__('motor_controller')
        self.get_logger().info("Initializing Phoenix Motor Controller...")

    # ----- MOTOR HARDWARE CONFIGURATION -----
```

```

        # If your robot spins when you tell it to go straight, or goes the wrong
way,
        # change these Booleans to True/False until it drives perfectly.
        self.swap_left_and_right = False # Set to True if J makes it turn Right
instead of Left
        self.invert_left          = False # Set to True if the Left wheel goes
backward when it should go forward
        self.invert_right         = False # Set to True if the Right wheel goes
backward when it should go forward
        # -----

        # Left Motor Driver (BTS7960)
        self.left_fwd = PWMOutputDevice(17)
        self.left_rev = PWMOutputDevice(27)

        # Right Motor Driver (BTS7960)
        self.right_fwd = PWMOutputDevice(25)
        self.right_rev = PWMOutputDevice(23)

        # Acceleration / Smoothing Configuration
        # 'step' is how much the speed can change every 0.05 seconds (the timer
rate).
        # A step of 0.05 means it takes 1.0 second to reach full speed from 0 to
1.0

        self.linear_step = 0.05
        self.angular_step = 0.05

        self.target_linear = 0.0
        self.current_linear = 0.0
        self.target_angular = 0.0
        self.current_angular = 0.0

        self.subscription = self.create_subscription(Twist, 'cmd_vel',
self.cmd_vel_callback, 10)
        self.timer = self.create_timer(0.05, self.control_loop) # 20Hz control
loop

    def approach_target(self, current, target, step):
        if current < target:
            return min(current + step, target)
        elif current > target:
            return max(current - step, target)
        return target

    def set_motor(self, fwd_pin, rev_pin, speed):

```

```

    speed = max(min(speed, 1.0), -1.0)
    if speed > 0:
        fwd_pin.value = speed
        rev_pin.value = 0.0
    elif speed < 0:
        fwd_pin.value = 0.0
        rev_pin.value = -speed
    else:
        fwd_pin.value = 0.0
        rev_pin.value = 0.0

def cmd_vel_callback(self, msg):
    # Update targets based on joystick/keyboard input
    self.target_linear = msg.linear.x
    self.target_angular = msg.angular.z

def control_loop(self):
    # Smoothly interpolate current speeds towards target speeds
    self.current_linear = self.approach_target(self.current_linear,
self.target_linear, self.linear_step)
    self.current_angular = self.approach_target(self.current_angular,
self.target_angular, self.angular_step)

    # Standard Differential Drive Kinematics using smoothed speeds
    left_speed = self.current_linear - self.current_angular
    right_speed = self.current_linear + self.current_angular

    # Apply hardware fixes if the physical wiring is swapped/reversed
    if self.swap_left_and_right:
        left_speed, right_speed = right_speed, left_speed
    if self.invert_left:
        left_speed = -left_speed
    if self.invert_right:
        right_speed = -right_speed

    max_speed = max(abs(left_speed), abs(right_speed))
    if max_speed > 1.0:
        left_speed /= max_speed
        right_speed /= max_speed

    self.set_motor(self.left_fwd, self.left_rev, left_speed)
    self.set_motor(self.right_fwd, self.right_rev, right_speed)

def main(args=None):
    rclpy.init(args=args)

```

```

node = MotorController()
try:
    rclpy.spin(node)
except KeyboardInterrupt:
    pass
finally:
    node.set_motor(node.left_fwd, node.left_rev, 0.0)
    node.set_motor(node.right_fwd, node.right_rev, 0.0)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Step 3: Register the Node and Build

Path: ~/ambers_ws/src/phoenix_control/setup.py Update the entry_points block:

Python

```

entry_points={
    'console_scripts': [
        'motor_controller = phoenix_control.motor_controller:main',
    ],
},

```

Build Command:

Bash

```

cd ~/ambers_ws
colcon build
source install/setup.bash
ros2 run phoenix_control motor_controller

```

11.3. Phase 3: Actuator, Sensor, and Communication Nodes

To bridge the gap between the hardware components and the ROS 2 Navigation stack, several dedicated hardware-interface nodes were developed to run on the Raspberry Pi 4 Model B.

11.3.1. Pump Controller Node (pump_controller.py)

This node controls the 24V water pump via an opto-isolated relay connected to GPIO 26. It subscribes to a custom boolean topic to trigger the physical fire suppression mechanism once the Nav2 target is reached.

Path: ~/ambers_ws/src/phoenix_control/phoenix_control/pump_controller.py

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import Bool
from gpiozero import OutputDevice
import time

class PumpController(Node):
    def __init__(self):
        super().__init__('pump_controller')
        self.get_logger().info("Initializing Phoenix Pump Controller...")

        # The relay for the 24V pump is connected to GPIO 26
        self.pump_relay = OutputDevice(26, active_high=True, initial_value=False)

        # Subscribe to a topic that signals when the Nav2 goal is reached
        self.subscription = self.create_subscription(
            Bool,
            'target_reached',
            self.target_reached_callback,
            10
        )

    def target_reached_callback(self, msg):
        if msg.data:
            self.get_logger().info("Target reached! Activating 24V water
pump...")
            self.activate_pump(duration=5.0) # Spray for 5 seconds

    def activate_pump(self, duration):
        self.pump_relay.on()
        time.sleep(duration)
        self.pump_relay.off()
        self.get_logger().info("Pump deactivated.")

def main(args=None):
    rclpy.init(args=args)
    node = PumpController()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.pump_relay.off() # Ensure pump is off on shutdown
        node.destroy_node()
        rclpy.shutdown()

```

```
if __name__ == '__main__':  
    main()
```

11.3.2. LiDAR Publisher Node (lidar_publisher.py)

This node reads the raw UART serial data from the Okdo LiDAR HAT, formats it, and publishes standard sensor_msgs/LaserScan messages required by the slam_toolbox and Nav2 stack.

Path: ~/ambers_ws/src/phoenix_control/phoenix_control/lidar_publisher.py

```
import rclpy  
from rclpy.node import Node  
from sensor_msgs.msg import LaserScan  
import serial  
import math  
  
class LidarPublisher(Node):  
    def __init__(self):  
        super().__init__('lidar_publisher')  
        self.get_logger().info("Initializing Okdo LiDAR Publisher...")  
  
        self.publisher_ = self.create_publisher(LaserScan, 'scan', 10)  
  
        # Standard UART setup for Raspberry Pi GPIO serial  
        self.serial_port = serial.Serial('/dev/ttyAMA0', baudrate=230400,  
timeout=1.0)  
  
        # Buffer to hold incomplete serial frames  
        self.serial_buffer = bytearray()  
  
        # Array to hold the most recent distance for 360 discrete degrees  
        self.current_ranges = [0.0] * 360  
  
        # Timer to read serial and publish at ~10Hz  
        self.timer = self.create_timer(0.1, self.publish_scan)  
  
    def publish_scan(self):  
        raw_data = self.serial_port.read_all()  
  
        if not raw_data:  
            return  
  
        # Decode the byte stream
```

```

ranges = self.parse_lidar_data(raw_data)

scan_msg = LaserScan()
scan_msg.header.stamp = self.get_clock().now().to_msg()
scan_msg.header.frame_id = 'lidar_link'
scan_msg.angle_min = 0.0
scan_msg.angle_max = 2 * math.pi

# 1 degree increment in radians
scan_msg.angle_increment = math.pi / 180.0

scan_msg.range_min = 0.35 # Calibrated to ignore robot chassis
scan_msg.range_max = 10.0 # 10m max distance
scan_msg.ranges = ranges

self.publisher_.publish(scan_msg)

def parse_lidar_data(self, raw_bytes):
    self.serial_buffer.extend(raw_bytes)

    # Process as long as we have at least one full packet
    while len(self.serial_buffer) >= 47:
        # Look for the header (0x54) and length (0x2C)
        if self.serial_buffer[0] == 0x54 and self.serial_buffer[1] == 0x2C:
            packet = self.serial_buffer[:47]

            # Start and End angles are in 0.01 degree units, little-endian
            start_angle = int.from_bytes(packet[4:6], byteorder='little') /
100.0
            end_angle = int.from_bytes(packet[42:44], byteorder='little') /
100.0

            # Calculate the angular step size between the 12 points
            diff = end_angle - start_angle
            if diff < 0:
                diff += 360.0
            step = diff / 11.0 # 12 points = 11 intervals

            # Extract the 12 distance measurements
            for i in range(12):
                point_angle = (start_angle + i * step) % 360.0

                # Each point is 3 bytes (2 distance, 1 intensity) starting at
index 6
                idx = 6 + i * 3

```

```

        distance_mm = int.from_bytes(packet[idx:idx+2],
byteorder='little')

        # Convert to meters
        distance_m = distance_mm / 1000.0

        # Map to the nearest integer degree index (0-359)
        degree_idx = int(round(point_angle)) % 360
        self.current_ranges[degree_idx] = distance_m

        # Pop the successfully parsed packet from the buffer
        self.serial_buffer = self.serial_buffer[47:]
    else:
        # If sync is lost, discard 1 byte and scan for the next valid
header
        self.serial_buffer.pop(0)

    return self.current_ranges

def main(args=None):
    rclpy.init(args=args)
    node = LidarPublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.serial_port.close()
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

11.3.3. MQTT to Nav2 Action Client (mqtt_nav_client.py)

Serving as the communication bridge in the split-node architecture, this node subscribes to local MQTT broker to receive target fire coordinates calculated by the global vision node. It then passes these coordinates to the Nav2 stack as an action goal.

Path: ~/ambers_ws/src/phoenix_control/phoenix_control/mqtt_nav_client.py

```

import os
import rclpy
from rclpy.node import Node

```

```

from rclpy.action import ActionClient
from nav2_msgs.action import NavigateToPose
from std_msgs.msg import Bool
import paho.mqtt.client as mqtt
import json

class MqttNavClient(Node):
    def __init__(self):
        super().__init__('mqtt_nav_client')
        self.get_logger().info("Initializing MQTT to Nav2 Bridge...")

        # ROS 2 Action Client for Nav2
        self.nav_client = ActionClient(self, NavigateToPose, 'navigate_to_pose')

        # Publisher to trigger the pump when goal is reached
        self.pump_trigger = self.create_publisher(Bool, 'target_reached', 10)

        # MQTT Setup
        # Using Callback API V2 to match the local node scripts
        self.mqtt_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
"Nav2_Client")
        self.mqtt_client.on_connect = self.on_connect
        self.mqtt_client.on_message = self.on_message

        # Connect to Local Broker
        # (Replace "localhost" with the Raspberry Pi's IP address if running on a
different PC)
        self.mqtt_client.connect("localhost", 1883, 60)

        # Start MQTT loop in the background
        self.mqtt_client.loop_start()

    def on_connect(self, client, userdata, flags, rc, properties):
        self.get_logger().info(f"Connected to Local Broker with result code
{rc}")
        client.subscribe("ambers/robot/navigation/target") # Update topic as
needed

    def on_message(self, client, userdata, msg):
        self.get_logger().info(f"Received MQTT Message: {msg.payload.decode()}")
        try:
            # Expecting JSON payload: {"x": 2.5, "y": 1.2} [cite: 481]
            data = json.loads(msg.payload.decode())
            target_x = float(data.get("x", 0.0))
            target_y = float(data.get("y", 0.0))

```

```

        self.send_nav_goal(target_x, target_y)
    except Exception as e:
        self.get_logger().error(f"Failed to parse MQTT message: {e}")

def send_nav_goal(self, x, y):
    self.get_logger().info(f"Sending Nav2 goal: x={x}, y={y}")
    self.nav_client.wait_for_server()

    goal_msg = NavigateToPose.Goal()
    goal_msg.pose.header.frame_id = 'map'
    goal_msg.pose.header.stamp = self.get_clock().now().to_msg()
    goal_msg.pose.pose.position.x = x
    goal_msg.pose.pose.position.y = y
    goal_msg.pose.pose.orientation.w = 1.0 # Assuming facing forward is fine

    self._send_goal_future = self.nav_client.send_goal_async(goal_msg)
    self._send_goal_future.add_done_callback(self.goal_response_callback)

def goal_response_callback(self, future):
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().info('Nav2 Goal rejected.')
        return
    self.get_logger().info('Nav2 Goal accepted, navigating...')
    self._get_result_future = goal_handle.get_result_async()
    self._get_result_future.add_done_callback(self.get_result_callback)

def get_result_callback(self, future):
    result = future.result().status
    if result == 4: # 4 corresponds to SUCCEEDED
        self.get_logger().info('Navigation Succeeded! Triggering Pump...')
        msg = Bool()
        msg.data = True
        self.pump_trigger.publish(msg)
    else:
        self.get_logger().info(f'Navigation failed with status: {result}')

def main(args=None):
    rclpy.init(args=args)
    node = MqttNavClient()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass

```

```
finally:
    node.mqtt_client.loop_stop()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

11.4: System Dependencies

- The complete ROS 2 Navigation 2 stack (ros-\$ROS_DISTRO-navigation2, ros-\$ROS_DISTRO-nav2-bringup, ros-\$ROS_DISTRO-nav2-msgs).
- The Paho-MQTT Python client library (pip install paho-mqtt).
- The Gazebo Harmonic to ROS 2 integration packages (ros-jazzy-ros-gz).
- The ROS 2 SLAM Toolbox (ros-\$ROS_DISTRO-slam-toolbox)

12. Current Progress and Future Work

12.1. Current Development Status

- **System Architecture & Workflow:** The data pipeline and communication flow have been successfully established. The system uses a distributed split-node model: a stationary laptop runs the custom CNN vision nodes to detect flames and prioritize human targets, and publishes coordinates to a local MQTT broker. The Raspberry Pi 4 Model B onboard the robot subscribes to this MQTT topic and routes the data to the Nav2 stack for autonomous driving.
- **Kinematic Adaptation:** The underlying mathematical model has been adapted to account for the physical hardware upgrade from a two-wheel to a four-wheel skid-steer configuration. Inverse and forward kinematics, including an empirical slip factor to manage lateral friction during turns, have been formalized.
- **Transformation Matrices Setup:** Internal spatial mapping (TF) linking the robot's base to the LiDAR, Pi Camera, and wheels has been documented . Furthermore, the methodology for real-time extrinsic tracking (mapping the surveillance camera's coordinate frame to the mobile robot) has been implemented using OpenCV, ArUco fiducial markers, and the solvePnP algorithm .
- **Testing Status:** Software development is actively progressing using a WSL environment on the laptop and the Raspberry Pi 4 Model B. Physical teleoperation

testing is temporarily paused until the complete 4-motor chassis is fully assembled and wired.

- **Global Vision Validation:** The extrinsic transformation tracking (mapping the surveillance camera's coordinate frame to the mobile robot) has been successfully tested and validated locally using OpenCV and ArUco markers.
- **Cloud Communication Bridge:** The MQTT-to-Nav2 bridge was successfully deployed to the Raspberry Pi 4 Model B. The hardware can securely connect to the local MQTT broker and parse JSON coordinate payloads asynchronously.
- **LiDAR Data Pipeline:** The UART serial decoding logic for the Okdo LD06 LiDAR has been fully implemented, converting 47-byte hex packets into standard ROS 2 LaserScan arrays.
- **Gazebo Harmonic Simulation Integration:** A full physics simulation of the Phoenix chassis has been successfully deployed using Gazebo Harmonic. The system accurately models skid-steer kinematics by implementing asymmetric lateral friction coefficients (μ_1 and μ_2), allowing the virtual robot to be teleoperated smoothly.
- **Virtual Sensor Bridging:** The onboard sensors (LiDAR and Camera) have been successfully virtualized and bridged into the ROS 2 ecosystem. To maintain stability within the WSL architecture, the rendering pipeline was forced to use the Ogre 1 engine, and the camera resolution was optimized to 640x480 at 15 FPS. These data streams are verified and viewable in standard ROS tools like `rqt_image_view`.
- **Autonomous Navigation Validation:** Successfully completed. The robot is now capable of computing collision-free paths around warehouse obstacles to a target coordinate using the Nav2 stack and a cleaned occupancy grid. The LiDAR self-detection issue was resolved by adjusting the minimum range parameters in the URDF and publisher nodes

12.2. Future Work Roadmap

To advance from the current foundational simulation and drivetrain nodes to full autonomy, the following milestones are scheduled:

1. **Teleoperation Testing:** Install `teleop_twist_keyboard` to send manual movement commands over Wi-Fi from the laptop to the Raspberry Pi. This will verify the wiring, PWM signal generation, and 4WD chassis traction on physical terrain .

2. **Sensor Node Integration:** publishing standard sensor_msgs/LaserScan messages. Establish a method to publish the robot's estimated odometry (odom to base_link transform).
3. **Fire Suppression Execution:** Develop a dedicated ROS 2 node to control the relay module responsible for triggering the 24V water pump. Implement logic that automatically triggers the pump to extinguish the flame once the Nav2 stack reports a successful arrival at the target coordinates.
4. **Autonomous Navigation Verification:** Load the generated warehouse_map into the ROS 2 Nav2 stack. Simulate a coordinate payload from the local broker and observe the robot autonomously computing a collision-free path around the warehouse shelves to the target red box.

13. Conclusion

The hardware and electrical system of the Intelligent Mobile Robot integrate high-performance computing, efficient power management, and robust actuation into a cohesive platform. By combining cloud-based vision processing with local autonomy, the robot achieves reliable fire detection and suppression while maintaining modularity and scalability. This architecture provides a strong foundation for future enhancements in navigation, perception, and industrial deployment.

References

1. Ren, X., Qu, K., Guo, J., Yin, H. and Huang, B., 2025. Research on accurate fire source localization and seconds-level autonomous fire extinguishing technology. *Scientific Reports*, 15(1), p.17135.
2. Yuvaraj, R., Bhalerao, S.A., Murugesan, K., Vellaiyan, S. and Van Minh, N., 2025. Real-Time Fire Detection and Suppression System Using YOLO11n and Raspberry Pi for Thermal Safety Applications. *Case Studies in Thermal Engineering*, p.107159.
3. Raspberry Pi Foundation, *Raspberry Pi 4 Model B Product Brief*, Raspberry Pi Ltd., 2023.
4. Raspberry Pi Foundation, *Raspberry Pi 4 Model B Datasheet and Hardware Documentation*, Raspberry Pi Ltd., 2023.
5. Sony Semiconductor Solutions Corporation, *IMX708 CMOS Image Sensor Datasheet*, Sony Corp., 2022.

6. Raspberry Pi Foundation, *Camera Module 3 Technical Documentation*, Raspberry Pi Ltd., 2023.
7. TP-Link Technologies Co., Ltd., *Tapo C210 Pan/Tilt Home Security Wi-Fi Camera Datasheet*, TP-Link, 2022.
8. XLSEMI, *XL4015 5A Adjustable Step-Down DC-DC Converter Datasheet*, XLSEMI Microelectronics, 2020.

Appendix A: Bill & Total Cost

Component	Description	Cost (EGP)
4x DC Motor	JGB37-545 High Torque Gear Motor 24VDC	2550
MG996R Servo Motor	Tower Pro Continuous Rotation (360°)	170
MG996R Servo Motor	Tower Pro 180 Degree	165
4x Robot Wheel	130mm + Metal Coupler 4mm	1500
Water Pump	24V Diaphragm Pump	500
Raspberry Pi Camera	Module V3 Official 12 MP and Autofocus IMX708	2750
Raspberry Pi 4 Model B	4GB RAM	6500

Component	Description	Cost (EGP)
Okdo LiDAR	LiDAR HAT For Raspberry Pi	4250
Camera	TP-Link Tapo C210 2K Pan/Tilt	945
2x Li-Po Battery	11.1 V, 5200 mAH, 40C	5000
Step Down (Logic)	XL4015 Adjustable DC-DC Step-Down 5A	140
7x Aluminum Profile	2020 V-Slot (black frame) 3m	1330
Water Tank	4L Capacity	35
Nozzle	Fire suppression nozzle	80
Water Pipe	Standard tubing	115
21x L Corner Mount	Bracket	525
210x T-Nut	Drop in for V-Slot 2020 Aluminum Bar (5mm)	735
16x T-Corner Mount	Bracket	400

Component	Description	Cost (EGP)
210x Screw	M5x10mm (Allen Head)	294
270x Washer	M5	108
2x Motor Driver	BTS7960 Single Channel 43A	900
3x Battery Male Connector	T-Type 15cm	150
Battery Female Connector	T-Type 15cm	50
Battery Charger	IMAX B6 Balancer Charger 80W	1650
Adapter Power Supply	12VDC – 3A	120
3D Printer Filament	Creality Hyper PLA, 1 kg, 1.75 mm	1440
28x Corner Mount	Corner 2020	660
3x Flange Bearing	KP08 Self-Aligning (Vertical - 8mm Dia)	180

Component	Description	Cost (EGP)
Camera Cable	Raspberry Pi 4 Model B Camera Ribbon FFC Adapter 300mm	225
Micro SD Card	64GB	480
SD Card Reader	Standard USB	20
Linear Shaft Rod	Stainless steel (8mm 1M)	150
2x Shaft End Support	SK8	130
Switch	Standard power switch	25
Raspberry pi cooler	Active cooler set	500
Fan	For cooling the electronic box	200
Encoder	IR Photoelectric encoder sensor	160
Encoder Disk	Encoder Disk	20
Total Spent:		35,152EGP

